SUP'COM

المدرسة العليا للمواصلات بتونس
ÉCOLE SUPÉRIEURE DES COMMUNICATIONS DE TUNIS

Telecommunications Engineering Degree

Major:

Applications des Informations Multimodales (AIM)

Final Graduation Project Report

Topic:

Estimate Dimensional Measures of an object from an image

Realized by:

Achraf Haddar

Academic supervisor:

Dr. Fethi Tlili

Professional supervisors:

Mr. Mohamed Ali Ben Jemaa

Ms. Fatma Abdelkefi

Dr. Inderjeet Singh

Work proposed and realised within: Infor

infor

Academic year : 2020-2021

Ecole Supérieure des Communications de Tunis
SUP'COM
2083 Cité Technologique des Communications - Elghazala - Ariana - Tunisie
Tél. +216 70 240 900 – site web : www.supcom.mincom.tn

Supervisors Signatures

Infor Supervisor:
Mr. Mohamed Ali Ben Jemaa

SUP'COM Supervisor:
Dr. Fethi Tlili

Acknowledgements

This report is the result of a pleasant graduation internship at Infor Company. That pleasure I owe to the interaction I have had with my supervisors, colleagues and co-workers.

A special gratitude I like to give to my supervisors Mr. Mohamed Ali Ben Jemaa, Ms. Fatma Abdelkefi and Dr. Inderjeet Singh for the patient guidance, encouragement and advice they have provided me during my internship. They always listen carefully to my questions and when I ran into a problem they guided me to resolve it.

A special thanks goes to Dr. Fethi Tlili, my supervisor at SUP'COM on his constant support, supervision and encouragement.

I would also like to express my deepest gratitude to all my teachers for their training during my study years.

I must express my very profound gratitude to my friends and my family for providing me with unfailing support and continuous encouragement throughout my years of study and for believing that I'm able to do it. This accomplishment would not have been possible without them.

Finally, I would like to thank the jury board members hoping that they will find in this report the expected clarity and motivation.

Thank you all



Abstract

Fishmeal nutritional contributions in aquaculture production cycles are being actively optimized by genetic teams at major fishing farms. In fact, aquaculture's role is critical in providing a nutritious, lean protein to the world's population in a sustainable manner. That's why their primary responsibility is to efficiently feeding the fish. Genetic teams keep track of the fish's growth characteristics throughout their lifecycle. Dimensional measurements, for example, length of the fish and weight are useful attributes that define growth. At present these dimensional measurements are done manually using tape measure and weight scale which is, however, time consuming and labour intensive. Therefore, the main idea of this project is to propose an automated method to determine the fork length and the weight of the fish from an image.

In this internship project, we first build an outlier detection model to ignore non-fish images from the rest of the process. Then, we implement two approaches to predict the fork length from the fish image: Image Processing approach which is based on object detection using traditional computer vision techniques and Machine Learning approach which is based on image segmentation and Convolutional Neural Network Regressor model. Next, we use segmented image from the ML approach to predict the gender of the fish using Convolutional Neural Network Classifier. Finally, we use the predicted fork length and sex to predict the weight of the fish using a tree based regression model.

Key words: Image segmentation, object detection, machine learning, regression, classification, deep convolutional neural networks, transfer learning.

Contents

Acknowledgements 3

Abstract 4

Contents 5

List of figures 8

List of tables 11

Glossary 12

Introduction 13

1 Project General Context 15
1.1 Introduction . . . 15
1.2 The Host Organization . . . 15
1.2.1 Overview of Infor . . . 15
1.2.2 Infor Decision Analytics and Science (IDeAS) . . . 15
1.3 Presentation of the Project . . . 16
1.3.1 Problem Statement . . . 16
1.3.2 Project Objectives and Needs . . . 17
1.4 Conclusion . . . 17

2 Literature Review 18
2.1 Introduction . . . 18
2.2 Principal Component Analysis . . . 18
2.3 Outlier Detection: One Class SVM . . . 20
2.3.1 Basic concepts of Support Vector Machines . . . 20
2.3.2 One Class SVM according to Tax and Duin . . . 21
2.4 Gradient boosting . . . 22
2.4.1 Ensemble learning . . . 22
2.4.2 Gradient Boosting algorithm . . . 23
2.4.3 eXtreme Gradient Boosting (XGBoost) . . . 24
2.5 Canny edge detection . . . 24
2.6 Deep Learning . . . 25
2.6.1 Artificial Neural Network . . . 26

2.6.2 Convolutional Neural Networks . . . 31
2.6.3 Transfer Learning . . . 34
2.6.4 Example of application field: Semantic Segmentation . 36
2.7 Conclusion . . . 38

3 Data and Problem Modeling **39**
3.1 Introduction . . . 39
3.2 Data . . . 39
3.2.1 Images . . . 39
3.2.2 Targets . . . 40
3.3 Solution Architecture . . . 40
3.4 Outlier detection . . . 42
3.4.1 Reason behind using outlier detection . . . 43
3.4.2 Preprocessing and Feature Extraction . . . 44
3.4.3 One Class SVM . . . 46
3.5 Fork Length prediction . . . 46
3.5.1 Target exploration . . . 46
3.5.2 Image Processing approach . . . 47
3.5.3 Machine Learning Approach . . . 52
3.6 Sex Prediction . . . 54
3.6.1 Target exploration . . . 54
3.6.2 Preprocessing and Training . . . 54
3.7 Weight Prediction . . . 55
3.7.1 Target Exploration . . . 56
3.7.2 Architecture . . . 56
3.7.3 Correlation With Target . . . 57
3.7.4 Training . . . 58
3.8 Implementation Details . . . 58
3.8.1 Hardware Environment . . . 58
3.8.2 Software Environment . . . 58
3.9 Conclusion . . . 59

4 Experimental Results **60**
4.1 Introduction . . . 60
4.2 Error Metrics . . . 60
4.2.1 Percentage Error . . . 60
4.2.2 Mean Absolute Percentage Error (MAPE) . . . 60
4.2.3 Accuracy and F1-score . . . 61
4.2.4 Dice Coefficient . . . 63
4.3 Outlier Detection . . . 64
4.3.1 settings . . . 64

4.3.2 Results 68
4.4 Fork Length prediction 69
4.4.1 Image Processing Approach 69
4.4.2 Machine Learning Approach 71
4.4.3 Final Evaluation 76
4.5 Sex Prediction 77
4.6 Weight prediction 80
4.6.1 Settings 80
4.6.2 Results 81
4.7 Conclusion 84

Conclusion and Perspectives **85**

Bibliography **87**

List of Figures

1.1 An image containing a fish. 17
1.2 Measuring manually length (Left) and weight (Right) of the fish. 17

2.1 Illustration of OC-SVM hypersphere[6]. 22
2.2 Biological neuron (left) vs. artificial neuron (right)[1]. 26
2.3 Three-layer feedforward neural network and node of the network. 27
2.4 Neuron with the function $z = f(x, y, \theta)$. 29
2.5 Curve of sigmoid function. 30
2.6 Curve of tanh function. 30
2.7 Curve of relu function. 31
2.8 An example of convolution operation with a kernel size of $3x3$[16]. 32
2.9 An example of convolution operation with a kernel size of $3x3$ with padding[16]. 33
2.10 Example of max-pooling filter[15]. 34
2.11 the performance parameters-accuracy curve for representative ConvNets trained on Imagenet[17]. 35
2.12 Residual block[15]. 36
2.13 U-net architecture (example for 32×32 pixels in the lowest resolution). Each blue box corresponds to a multi-channel feature map. The number of channels is denoted on top of the box. The x-y-size is provided at the lower left edge of the box. White boxes represent copied feature maps. The arrows denote the different operations[14]. 38

3.1 An image example from the dataset. 39
3.2 Global architecture. 42
3.3 Examples of non-fish images. 43
3.4 Number of occurrences for fish and non-fish images. 43
3.5 Transforming an image to a feature vector. 45
3.6 Distance of fork length. 46
3.7 Distribution of the fork length. 47
3.8 Image processing approach. 48
3.9 Difference between background and not background pixels. . . 49
3.10 output image after applying resizing and removing the background steps. 49
3.11 output image after applying thresholding. 50

3.12 Detected edges using Canny Edge Detection algorithm. 50
3.13 Output image after applying closing filter. 51
3.14 Detected objects in the image. 52
3.15 Machine Learning approach. 52
3.16 Created mask using Labelme tool. 53
3.17 Number of occurrences for Male and Female fishes. 54
3.18 Visual explanation of over-sampling. 55
3.19 Distribution of the weight. 56
3.20 Architecture for weight prediction. 57
3.21 Correlation between the fork length, the sex and the weight. . 57

4.1 Visual explanation of MAPE. 61
4.2 Confusion Matrix for binary classification. 62
4.3 Illustration of Dice Coefficient. 64
4.4 Example of extracted feature vector. 65
4.5 Decision boundary for One-Class SVM with kernel="linear". . 66
4.6 Decision boundary for One-Class SVM with kernel="poly". . . 66
4.7 Decision boundary for One-Class SVM with kernel="rbf". . . . 67
4.8 Variation of validation's F1-score for different nu and gamma combinations. 68
4.9 Confusion matrix for test dataset. 69
4.10 Actual vs Predicted Fork Length with Image Processing Approach for 50 samples. 70
4.11 A visual representation of predicted and actual fork length in a fish image example. 71
4.12 Unwanted objects detected in images. 71
4.13 Training and validation loss curves 72
4.14 Examples of predicted masks for fishes looking to the left . . . 73
4.15 Examples of predicted masks for fishes looking to the right . . 73
4.16 Examples of predicted masks 74
4.17 Example of a segmented image 74
4.18 Actual vs Predicted fork length for train dataset. 75
4.19 Actual vs Predicted fork length for test dataset. 76
4.20 Predictions for 50 samples with Machine Learning and Image Processing Approach 77
4.21 Train accuracy. 79
4.22 Validation accuracy. 79
4.23 Confusion Matrix for test dataset. 80
4.24 Comparison between Gradient Boosting, LightGBM and XGBoost models. 82
4.25 Actual vs Predicted weight for train dataset. 83

4.26 Actual vs Predicted weight for test dataset. 83

List of Tables

3.1 Subsample of the metadata 40
3.2 Subsample of the targets dataframe 40
3.3 Hardware Characteristics 58

4.1 Splits of the dataset. 64
4.2 F1-score for validation dataset for different Nu and Gamma combinations 68
4.3 Experiment's specifications and results. 72
4.4 Experiment's specifications and results. 78
4.5 Results of experiments. 81

Glossary

ANN: Artificial Neural Network

MSE: Mean Square Error

BCE: Binary Cross Entropy

CCE: Categorical Cross Entropy

SVDD: Support Vector Data Description

SVM: Support Vector Machine

MAE: Mean Absolute Error

MAPE: Mean Absolute Percentage Error

General Introduction

Because of its high protein content, fish is a vital nutritional component. Fishermen should gather useful information such as length and weight in order to sustainably manage fish resources with high efficiency.

Fish size is an important piece of data that may be used to assess stock status, provide management advice, and improve the economic benefits of an aquaculture company. Fish length information is used to define and monitor fish growth so that the stock can be harvested at the best time. Additionally, length information can be used to evaluate quality in the food business, which can aid fishermen in determining the price. The weight of the fish is also a significant and direct criterion for determining whether the fish is satisfactory.

Given the importance of fish's length and weight, fishers measure those dimensions by traditional measurement methods. Fishers sometimes estimate length by eye based on experience. However, they often use a ruler. To measure the weight, fishers place the fish on a weight scale. These techniques are time-consuming, labor-intensive, as well as costly to operate. As a result, a more straightforward, quick, and scientific approach for gathering the length and the weight of the fish is required.

Various industries have created and implemented machine vision systems which can record images in real time, analyze them, and make decisions. In addition, machine vision has been widely employed in aquaculture for a variety of purposes, including health monitoring, illness diagnosis, behavior detection, species identification, etc...

Image-based methods have been applied for measuring the length of the fish. Shafry, Rehman, Kumoi, Abdullah, and Saba [12] (2012) determined the length of harvested fish from images using optical theory, geometry concepts, and image processing techniques.

Other approaches have sought employing convolutional neural networks to detect and locate the fish heads and tails forks and use it to estimate the body lengths. Tseng, Hsieh and Kuo [3] (2020) determined the snout-to-fork lengths of harvested fish using a fully automatic approach. In this approach, they collected images of fish bodies and colour plates (with known dimension)

placed beside. A convolutional neural network classifier was then developed to automatically detect the locations of the regions of fish head, tail fork and colour plate. The snout and fork points of the fish and the corners of the colour plates were then determined using image processing. The length of the fish was estimated using the distance between the snout and fork points and the length-to-pixel ratio obtained from the colour plate.

Instead of detecting separately the fish head and the tail fork, in this project, we are building a pipeline for automating fork length, gender and weight estimation of a fish from an image based on detecting and localizing the entire fish.

The report is organized as follows:

- In the first chapter, we describe the general frame of the project by introducing the host company then presenting the project overview.
- Throughout the second chapter, we present the state of the art and explain some important concepts in Principal Component Analysis, outlier detection, gradient boosting, canny edge detection and deep learning. Those concepts will help to clarify the understanding of the following chapters.
- In the third chapter, we describe the data used in the scope of this project and we present the adopted methods and architectures for estimating the fish's fork length, sex and weight. Finally, we will end the chapter presenting the technologies and the programming languages used in conducting the work.
- The fourth and last chapter, we present the obtained results of different experiments and their interpretation.

Chapter 1
Project General Context

1.1 Introduction

This chapter is composed of two principal sections. In the first section, we present an overview of the hosting company, Infor, in which this project was carried out. In the second section, we introduce the general context of the project and the main problem that we want to solve.

1.2 The Host Organization

1.2.1 Overview of Infor

Infor is a global enterprise leader in business cloud software, headquarted in New York City, United States. Infor employs over 17.000 people globally with 168 offices located all over the world. Infor builds enterprise software solutions for the industry and delivered via the cloud as a service. From Manufacturing to Healthcare to supply chain and customer relationship management, Infor covers all industries and have (ERP) end-to-end solutions that are brought together by artificial intelligence technology.

Infor creates cloud-based industry suites and effectively deploys technology that prioritizes the user experience. It offers solutions for forecasting, planning, assortment, pricing, promotions, and replenishment, etc. Infor business applications are specialized by industry and designed for the cloud to allow a global supply chain, networked analytics, and an artificial intelligence-led user interface to drive continuous innovation in the enterprise.

1.2.2 Infor Decision Analytics and Science (IDeAS)

This internship is held within the Infor Decision Analytics and Science (IDeAS) team which is part of the bigger group Applied Innovation. IDeAs team is responsible for setting innovation strategy, enabling and delivering customized solutions that are powered by ML and AI algorithms for Infor customers across multiple segments and industries. Besides, IDeAS focuses on using entreprise's dataset to improve its performance.

IDeAS group is composed of Product Managers, Data Engineers, Solution Architects, and Data Scientists to generate various value driven models across industries.

- **Science team**
  Science team in IDeAS is the brain behind every data-driven decision analytics. Data scientists design, create and operationalize large scale predictive and prescriptive analytics models to solve business problems.
- **Engineering team**
  Engineering team in IDeAS rely on a combination of analytics and technical expertise to turn the data vision into reality.
- **Decision Intelligence Product Management team**
  Decision Intelligence Product Management team is responsible of defining product assets vision, managing the execution and market the output.

IDeAS team members are engaged with customers to solve their specific business problems in a course of client engagement.

1.3 Presentation of the Project

1.3.1 Problem Statement

The most important job for a fish farmer is efficient feeding of the fish. Feed it too little as a result "leaving growth on the table". Feed it too much and not only wasting most expensive input factor, but also polluting. This implies there is an optimal feeding point.

Genetic teams at major fishing farms actively work on optimizing nutritional contributions of fishmeal in aquaculture production cycles. These teams track growth traits throughout the lifecycle of the fish. Dimensional measurements, for example, length of the fish, surface area and weight are useful attributes that define growth.

At present these dimensional measurements are done manually (Figure 1.2) and the objective is to automate a section of this data collection procedure by creating a prediction model that can predict the fork length and weight of the fish from an image. Figure 1.1 shows an example of an image containing a fish.



Figure 1.1: An image containing a fish.



Figure 1.2: Measuring manually length (Left) and weight (Right) of the fish.

1.3.2 Project Objectives and Needs

In this project, as we specified above, the main objective is to automate the process of measuring the dimensions of a fish. This project aims to model and implement a solution that uses the image of a fish to predict the fork length, the sex and the weight

1.4 Conclusion

This chapter is dedicated to describe the host company and the project context. The next chapter, state of the art, we provide a literature review to establish the theoretical context of the present work.

Chapter 2
Literature Review

2.1 Introduction

This chapter summarizes the relevant fundamentals needed to implement the proposed work. In section 2.2, we introduce the basic concepts of Principal Component Analysis. Then, in sections 2.3, 2.4 and 2.5 we expose briefly the main basics of algorithms used for outlier detection, gradient boosting and edge detection. Finally, in chapter 2.6 we present the basics of Deep Learning and Convolutional Neural Network and the architecture used to perform semantic segmentation.

2.2 Principal Component Analysis

Principal Component Analysis[9], or PCA, is a technique that reduces the dimensionality of large data sets by converting a large number of features into a smaller set that retains the majority of the information in the larger set. In fact, smaller datasets are simpler to explore and visualize, and they are easier and quicker to analyze by machine learning algorithms.
PCA is a multi-step algorithm:

- **Step 1: Standardization**
  In this step, we standardize continuous features in the dataset so that they will have the same contribution to the analysis.

  If the ranges of initial variables vary significantly, the variables with greater ranges would dominate over those with smaller ranges, resulting in biased results. Features with larger ranges would dominate over those with smaller ranges and as a result we obtain biased predictions. So, we should transform the data to comparable scale to prevent this problem.

  To apply standardization, we need to substract the mean and divide by the standard deviation for each value of each feature as shown in equation 2.1.

$$x_{scaled} = \frac{x - \nu}{\sigma}, \tag{2.1}$$

where: ν and σ are respectively the mean and standard deviation of x.

- **Step 2: Covariance Matrix Computation**
  We aim in this step to figure out if there is a relationship between different features since they can sometimes be strongly correlated in such a way that they contain redundant information. We use covariance matrix to find these correlations.

  The covariance matrix is a $p \times p$ (where p is the number of dimensions) symmetric matrix with the covariances associated with all possible pairs of the features as entries.
  We have the variances of each variable in the main diagonal since the covariance of a variable with itself is equal to its variance.

  The entries of the covariance matrix are symmetric with respect to the main diagonal.

- **Step 3: Compute the eigenvectors and eigenvalues of the covariance matrix to identify the principal components**
  We constract principal components which are new variables using linear combinations in such a way that they are uncorrelated and the first components contain the majority of the information from the initial variables.

  The data directions that describe the highest amount of variance, or the lines that represent the most information in the data, are known as principal components.

  Eigenvectors of the Covariance matrix are principal components and the coefficients attached to them are eigenvalues which give the sum of variance carried in each principal component.

  We obtain the principal components in order of significance by ranking the eigenvectors in order of their eigenvalues, from highest to lowest.

- **Step 4: Feature Vector**
  In this step, we determine whether to maintain all of the components obtained in the previous step or to discard those with low eigenvalues, and build a matrix of vectors called Feature Vector with the remaining ones. The feature vector is just a matrix with the eigenvectors of the components we want to hold as columns.

- **Step 5: Recast the data along the principal components axe**
  In this last step, we aim to use the feature vector to reorient the data from

the original axes to the ones represented by the principal components. This is done by multiplying the transpose of the original data set by the transpose of the feature vector as described in equation (2.2).

$$Final\ Dataset = Feature\ Vector^T \times Standardized\ Original\ Dataset^T \tag{2.2}$$

2.3 Outlier Detection: One Class SVM

The goal of machine learning classification models is to distinguish test data between a number of classes greater than two, using training data. However, in some cases, we may have only one class in the training dataset and the goal is to test new data and found out whether it is alike or not like the training data. This problem is known as anomaly detection problem where we would like to determine whether new test data is member of a specific class, determined by our training data, or is not. One-Class Support Vector Machine (OC-SVM) is one of the state-of-the-art approaches for anomaly detection[20].

2.3.1 Basic concepts of Support Vector Machines

In this subsection, we introduce the traditional two-class support vector machine.

Consider a dataset $\Omega = (x_1, y_1), (x_2, y_2), \ldots, (x_n, y_n)$ where $x_i \in \mathbb{R}^d$ refers to the i-th input data point and $y_i \in \{-1, 1\}$ refers to the i-th output, indicating the class membership.

SVMs can create a non-linear decision boundary by projecting data into a higher-dimensional space using a non-linear function ϕ. SVMs use this projection when data points cannot be separated by a straight line in their original space I. In such case, SVMs use projection to "lift" data points to a feature space F where there can be a "straight" hyperplane that separates them of one class from an other. The hyperplane takes the shape of a non-linear curve when projected back to the input space I.

The hyperplane searches for the maximal margin between the classes. The hyperplane's equation is defined by equation (2.3).

$$w^T x + b = 0, \tag{2.3}$$

where $w \in F$ and $b \in \mathbb{R}$

SVM classifier avoids over-fitting with noisy data using slack variables ξ_i and

constant $C > 0$. ξ_i introduced to allow some data points to lie within the margin and constant C determines the trade-off between maximizing the margin and the number of training data points within that margin.

The objective function of the SVM classifier is shown in equation (2.4).

$$\begin{gathered} \min_{w,b,\xi} \frac{\|w\|^2}{2} + C \sum_{i=0}^{n} \xi_i \\ \text{subject to:} \\ y_i(w^T \phi(x_i) + b) \geq 1 - \xi_i \text{ for all } i = 1, ..., n \\ \xi_i \geq 0 \text{ for all } i = 1, ..., n \end{gathered} \tag{2.4}$$

We use Lagrange multipliers to solve this minimization problem. The decision function $f(x)$ of a data point x is shown in equation (2.5).

$$f(x) = sgn(\sum_{i=1}^{n} \alpha_i y_i K(x, x_i) + b), \tag{2.5}$$

where α_i are the Lagrange multipliers.

The function $K(x, x_i)$ is known as the kernel function. Popular choices for the kernel function are linear, polynomial but mostly the Gaussian Radial Base Function. The RBF's equation is shown in equation (2.6).

$$K(x, x_i) = \frac{\exp(-\|x - x_i\|^2)}{2\sigma^2}, \tag{2.6}$$

where $\sigma \in \mathbb{R}$ is a kernel parameter and $\|x - x_i\|$ is the dissimilarity measure[20].

2.3.2 One Class SVM according to Tax and Duin

In OC-SVM[18], the algorithm creates a spherical boundary, in feature space, around the data. This hypersphere has a center a and a radius $R > 0$ which is the distance between the center and (any support vector on) the boundary.

The center a is formed by linearly combining the support vectors that correspond to the training data points for which the Lagrange multiplier is non-zero. To create a soft margin, we introduce again slack variables ξ_i and a parameter C. The minimization problem is as follows:

$$\begin{gathered} \min_{R,a} R^2 + C \sum_{i=1}^{n} \xi_i \\ \text{subject to:} \\ \|x_i - a\|^2 \leq R^2 + \xi_i \text{ for all } i = 1, ..., n \\ \xi_i \geq 0 \text{ for all } i = 1, ..., n \end{gathered} \tag{2.7}$$

To solve this minimization problem, we use Lagrange multipliers α_i. By using the Gaussian kernel as a distance function over two data points 2.8, we can test a new data point z to be in or out of class based in its distance to the center a.

$$\|z - x\|^2 = \sum_{i=1}^{n} \alpha_i \exp(\frac{-\|z - x_i\|^2}{\sigma^2}) \geq -\frac{R^2}{2} + C_R \tag{2.8}$$

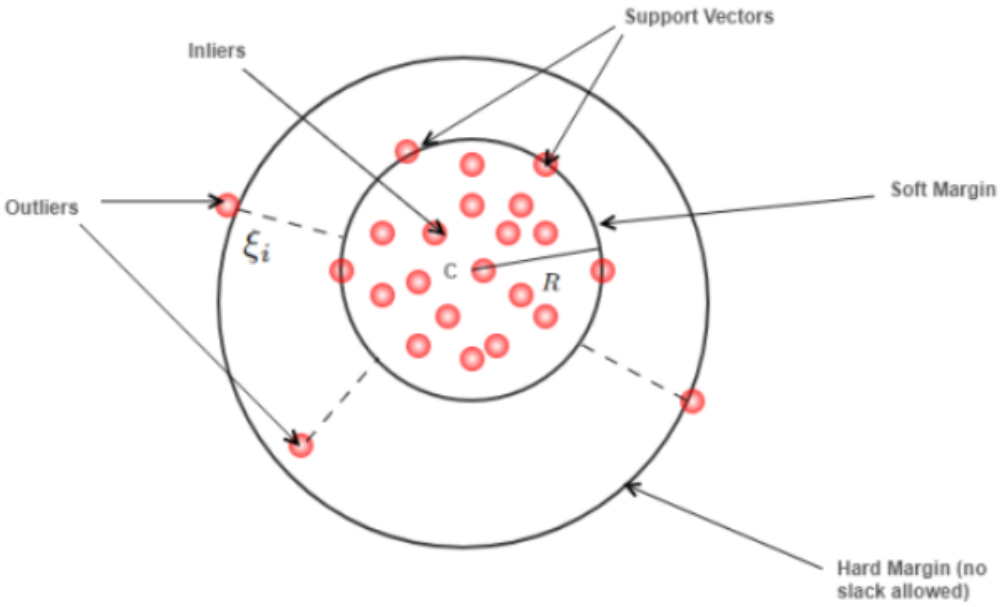


Figure 2.1: Illustration of OC-SVM hypersphere[6].

- If the distance of the data point from center $a > R$: outlier.
- If the distance of the data point from center $a < R$: inlier.
- If the distance of the data point from center $a = R$: support vector.

2.4 Gradient boosting

2.4.1 Ensemble learning

When using any machine learning model to predict the target variable, the main causes of difference between actual and predicted values are noise, variance, and bias. Ensemble aids in the reduction of these factors (except noise, which is irreducible error).

An ensemble is simply a group of predictors that are combined (e.g., the

mean of all predictions) to produce a final prediction. We use ensembles since several different predictors, attempting to predict the same target variable, would do better than any single predictor alone.

Boosting[8] is an ensemble technique in which the predictors are made sequentially. The logic behind this technique is that subsequent predictors learn from the errors of previous predictors. Predictors may be selected from a variety of models, such as decision trees, regressors, and classifiers. Since new predictors learn from previous predictor's errors, it takes less time and less iterations to get close to actual predictions. Gradient Boosting is an example of boosting algorithm.

2.4.2 Gradient Boosting algorithm

Gradient boosting[10] is a machine learning technique for regression and classification problems that generates a prediction model from an ensemble of weak prediction models, usually decision trees.

This section is used to explain the steps of gradient boosting algorithm for regression problem. In fact, gradient boosting is mainly composed of 7 steps:

- **Step 1: Calculate the average of the target label**
  When dealing with regression problems, we begin with a leaf, which represents the average value of the variable to predict.
- **Step 2: Calculate the residuals**
  For every sample, we calculate the residual with the formula (2.9).

  $$residual = actual\ value - predicted\ value, \tag{2.9}$$

  where *predicted value* is equal to the mean calculated in the previous step.
- **Step3: Construct a decision tree**
  In this step, we build a tree with the goal of predicting the residuals. In other words, every leaf will contain a value equal to the value of the residual.

  In the case where there are more residuals than leaves, some residuals will end up inside the same leaf. When this happens, we compute their average and place that inside the leaf.
- **Step 4: Predict the target label using all of the trees within the ensemble**

Each sample passes through the newly developed tree's decision nodes before it hits a specific lead. In order to prevent overfitting, a hyperparameter called learning rate is introduced. Every residual is multiplied by the learning rate when we make a prediction.

- **Step 5: Compute the new residuals**
  By subtracting the actual target values from the predictions made in the previous stage, we get a new collection of residuals. As defined in step 3, the residuals will be used for the leaves of the next decision tree.
- **Step 6: Repeat steps 3 to 5**
  In this step, we repeat steps 3 to 5 until the number of iterations matches the number specified by the hyperparameter which is the number of estimators
- **Step 7: Make final prediction**
  Once trained, we use all of the trees in the ensemble to make a final prediction. The final prediction will be equal to the first step's mean plus all of the residuals expected by the forest's trees multiplied by the learning rate.

2.4.3 eXtreme Gradient Boosting (XGBoost)

The eXtreme Gradient Boosting (XGBoost)[5] algorithm is a scalable and improved version of the gradient boosting algorithm that is optimized for effectiveness, computational speed, and model performance. XGBoost is a combination of software and hardware technologies designed to improve existing boosting techniques with precision and speed. XGBoost provides:

- **Parallelization** of tree construction using all of CPU cores during training.
- **Distributed Computing** for training very large models using a cluster of machines.
- **Out-of-Core Computing** for very large datasets that don't fit into memory.
- **Cache Optimization** of data structures and algorithm to make the best use of hardware.

2.5 Canny edge detection

In 1983, MIT's John Canny introduced the canny edge detection technique. It is the most popular and widely used edge detection tool. Canny is a better

method for extracting edges than other methods currently available, and it yields decent results. The Canny operator can manipulate a variety of edge picture data and effectively suppress noise.[11]

It is a multi-step algorithm:

- **Step 1: Smoothing the image**
  Since edge detection is sensitive to image noise, the first step is to use a Gaussian filter to eliminate high-freq noises from the image.
- **Step 2: Image gradient**
  The image is then smoothed and filtered with a Sobel filter in the horizontal and vertical axis to obtain the first derivative in the horizontal direction (Gx) and vertical direction (Gy). The edge gradient can be found by comparing these two images.
- **Step 3: Non-maximum suppression**
  After determining the gradient's direction, the image is thoroughly examined to eliminate any unnecessary pixels that may not be part of the edge. To accomplish this, each pixel is compared to its two neighbors. If the compared pixel is greater than the neighbor, the pixel is not changed. Otherwise, the pixel is not the maximum value, and the zero is assigned to it.
- **Step 4: Tracking the edge by hysteresis**
  This phase determines which edges are true edges and which are not. We need two threshold values, minVal and maxVal, for this. Edges with an intensity gradient greater than maxVal are likely to be edges, whereas those with an intensity gradient less than minVal are not certain to be edges and are thus rejected. According to their connectivity, those that fall between these two levels are classified as edges or non-edges. They are considered to be part of the edges if they are connected to pixels on the "sure edge". Otherwise, they are also rejected.

2.6 Deep Learning

Deep learning is a type of machine learning that uses a complex network of interconnected neural networks. It processes large amounts of data to identify common rules and characteristics.

To understand what deep learning is, we must first introduce the ANN, then the different types of layers and techniques used to improve training performance. Finally, we examine some of the models used in this project.

2.6.1 Artificial Neural Network

Artificial Neural Networks, or ANNs, are computer programs inspired by the structure of the biological brain. The human brain is made up of billions of neurons that interpret sensory information such as vision, touch, and acoustics. The connections between neurons are known as synapses.

A single neuron receives several inputs from previous neurons, known as dendrites. The neuron "fires" its single output through the axon after processing the inputs. Following the axon, the output will be sent to all linked neurons.

Each artificial neuron takes input from its dendrites and outputs along its (single) axon. The axon eventually spreads out and forms connections with the dendrites of neighboring neurons.

On each artificial neuron, there are n input connections. The weighted sum, a bias b, and an activation function (which will be discussed later) are used by the neuron to interpret the inputs: $f(\sum_{i=0}^{n} \theta_i x_i + b)$. The perceptron is composed input layer, hidden layer and output layer. Figure 2.2 illustrates the parallels of a biological and an artificial neuron.

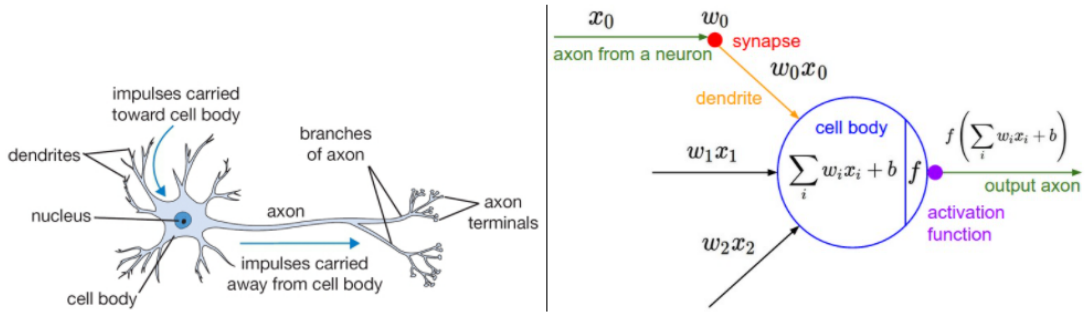
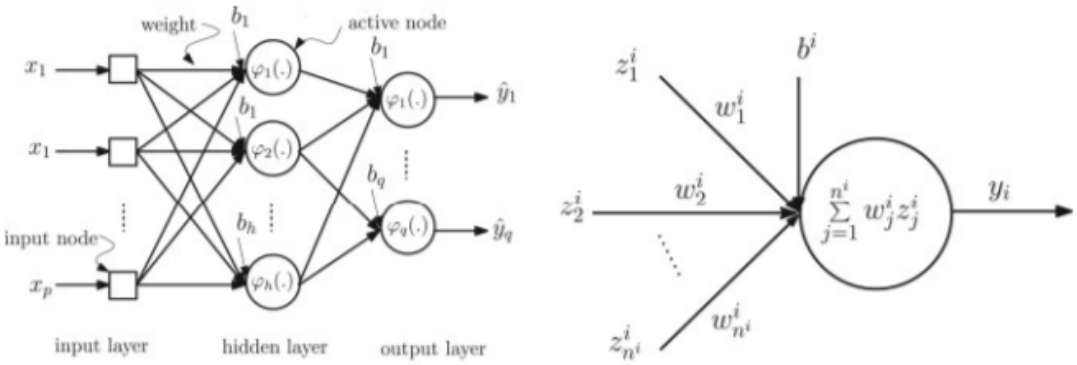


Figure 2.2: Biological neuron (left) vs. artificial neuron (right)[1].

Neural networks approximate a non-linear function $f(x)$ by adjusting θ_i of all neurons. Learning is the process of finding a successful parameter set θ that contains all weights θ_i.

A feedforward Neural Network organizes the artificial neurons in different layers. Each network has one input layer, that processes the raw input data, and one output layer, that contains the approximated result. In between those two layers can be one or more hidden layers where the relevant computing is happening. Figure 2.3 shows a Fully-connected, feedforward Deep Neural Network. Since all neurons of the outgoing layer are connected to all neurons of the incoming layer, it is a fully-connected layer.[7].



(a) Three-layer feedforward neural network (b) Node of the network

Figure 2.3: Three-layer feedforward neural network and node of the network.

Learning Process

The goal of the learning process is to determine the best parameter set for approximating the function $f(X)$. The true output Y of a given input X is used to update the parameters. It is an iterative process with the steps outlined below.

- **Forward Pass:**
  A fully connected layer's forward pass consists of one matrix multiplication, a bias offset, and an activation function. The input X is sent through the network, and the expected output Y_{pred} is obtained $Y_{pred} = f(X, \theta)$.

$$Y_{pred} = \sum (weight \times input) + bias \tag{2.10}$$

- **Loss Function:**
  We compare the expected output Y_pred to the real output Y using the loss $L(\theta)$ also known as the cost function and is chosen depending on the learning task.
  - **Mean Square Error[13]:** One of the most commonly used loss function for regression task.

$$L_{MSE} = \frac{1}{n}\sum_{i=1}^{n}(y_i - y_{pred_i})^2 \tag{2.11}$$

  - **Binary Cross-Entropy Loss[2]:** is a loss function that is used in binary classification problems. The binary cross entropy measures the difference between the prediction and the true value (which is

either 0 or 1).

$$L_{BCE} = -(Y \log(p) + (1 - Y) \log(1 - p)) \tag{2.12}$$

– **Categorical Cross-Entropy Loss[2]:** In multi-class classification tasks, we use categorical cross entropy as a loss function..

$$L_{CCE} = -\sum_{c=1}^{M} y_{o,c} log(p_{o,c}), \tag{2.13}$$

where:
M - number of classes.
p - predicted probability observation o is of class c.

- **Back-propagation:**
The network's global gradient of loss, $\nabla L(\theta)$, is computed and backpropagated. The back-propagation algorithm provides all hidden neurons with a local gradient of loss. The chain rule multiplies local derivatives to calculate derivatives of a composed function. Given the function $y = g(x)$ and $z = f(g(x))$, the $\frac{\partial z}{\partial x}$ derivative can be computed according to the equation (2.14).

$$\frac{\partial z}{\partial x} = \frac{\partial z}{\partial y} \frac{\partial y}{\partial x} \tag{2.14}$$

To propagate the global gradient loss $\frac{\partial L(\theta)}{\partial \theta}$ back through the network, the chain rule is used. It flows in the opposite direction of the forward pass.

Figure 2.4 represents a neuron with the $z = f(x, y, \theta)$ function. Its local derivatives are $\frac{\partial z}{\partial x}$, $\frac{\partial z}{\partial y}$ and can be determined during the forward pass. Back-propagation produces the local gradient of loss by multiplying the local derivative with the local gradient of loss of the next layer's associated neuron $\frac{\partial L}{\partial z}$. As a consequence, for each neuron's input, a local gradient of loss is obtained: $\frac{\partial L}{\partial x}$, $\frac{\partial L}{\partial y}$. They would then be back-propagated to the neurons that came before them. The gradients are simply added up if a neuron is linked to multiple neurons in the next layer.

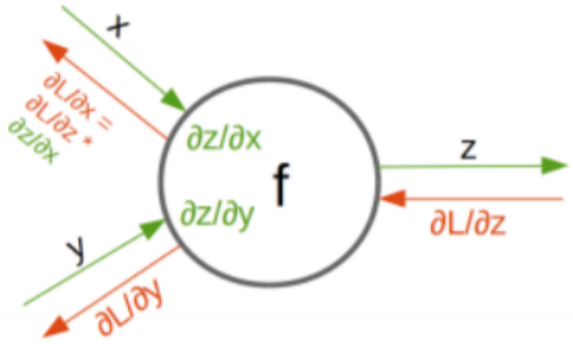


Figure 2.4: Neuron with the function $z = f(x, y, \theta)$.

- **Update:**
  In this step, the weights of all neurons are updated using an optimizer. A common optimizer is Adam optimizer which combines batch learning with gradient descent. Gradient descent tries to change the weights in the negative direction of the gradient of loss in each iteration in such a way the function approximation is taken to its minimum. A local minimum is supposed to be achieved after a number of iterations. The gradient descent update rule is shown in equation (2.15).

$$\theta_i \leftarrow \theta_i + \alpha \nabla_\theta L(\theta), \tag{2.15}$$

  where α is the learning rate parameter, which specifies how fast the minimum should be reached. If the learning rate α is too high, there is a risk that the minimum cannot be reached, because the taken steps are too big and will be overshot.

Activation function

There are three widely used activation functions: sigmoid, tanh and ReLU.

- **Sigmoid[19]:**
  The sigmoid function, shown in equation (2.16), transforms the input value x into a number between 0 and 1. Big negative values are turned into 0 and large positive values are turned into 1.

$$sigm(x) = \frac{1}{1 + \exp(-x)} \tag{2.16}$$

  The curve of sigmoid function is shown in figure 2.5

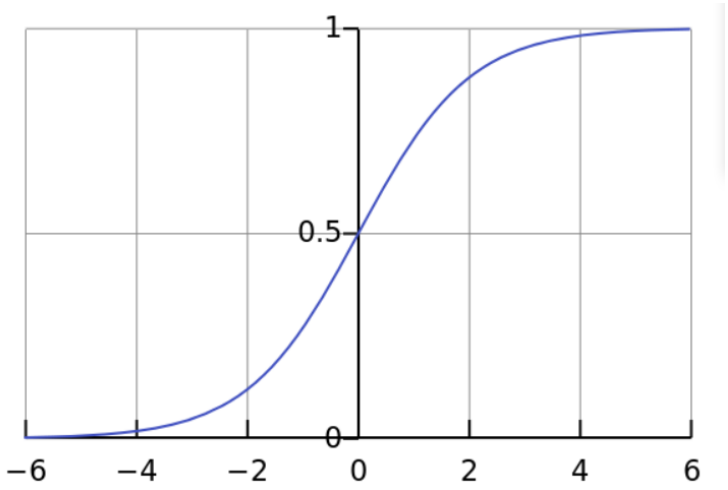


Figure 2.5: Curve of sigmoid function.

- **Tanh[19]:**
  The tanh function, shown in equation (2.17), zero-centers the sigmoid function.

$$tanh(x) = 2sigm(2x) - 1 \tag{2.17}$$

  The curve of tanh function is shown in figure 2.6.

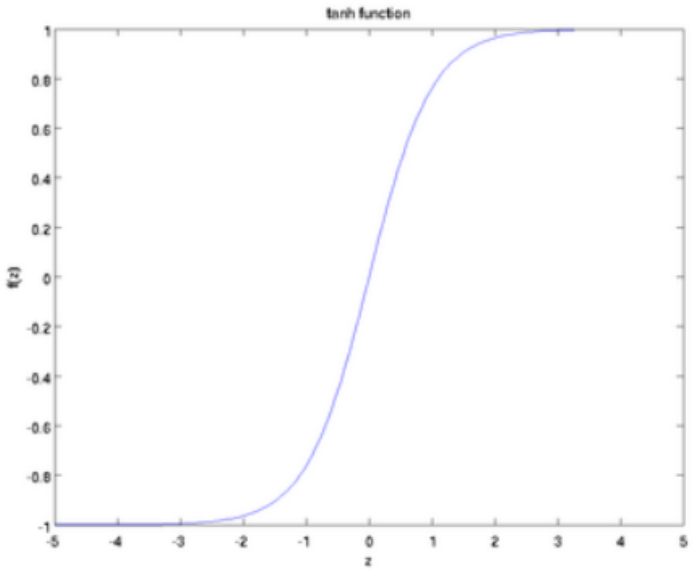


Figure 2.6: Curve of tanh function.

- **ReLU[19]:**
  The ReLU function, shown in equation (2.18), is the most popular function. It prevents the output from dropping below zero. In addition, it has a non-saturating shape, allowing the gradient to converge more quickly

during training. Another advantage is that it is a simple function with a small computational cost.

$$ReLU(x) = max(0, x) \tag{2.18}$$

The curve of ReLU function is shown in figure 2.7

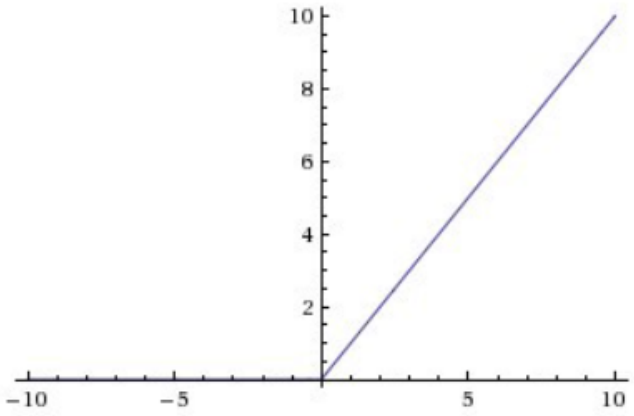


Figure 2.7: Curve of relu function.

Batch learning

Instead of processing a single training example, Batch Learning processes a collection of m training samples (mini-batches). The gradient is calculated by averaging all m processed training examples. This can result in a more reliable gradient with less variation, as well as a shorter training time.

2.6.2 Convolutional Neural Networks

Since they efficiently manage massive input data, Convolutional Neural Networks, or CNNs, are extensively employed in state-of-the-art approaches in the fields of Computer Vision, such as object recognition and image segmentation.

A standard CNN's architecture includes stacks of Convolutional Layers followed by a subsampling Pooling Layer. Low-level features such as edges and corners are learned in the early stages of the network, and these features are combined to form high-level features in later layers.

Convolutional Layer

A convolutional layer uses a mixture of linear and nonlinear operations, such as convolution and activation functions, to extract features.

Convolution is a type of linear operation for feature extraction that applies a small array of numbers called kernel across the input, which is a tensor of numbers. An element-wise product between each element of the kernel and the input tensor is determined and summed at each point of the tensor to give the output value at the corresponding position of the output tensor, referred to as a feature map. This process is repeated with different kernels to build an arbitrary number of feature maps that reflect the different characteristics of the input tensor. As a result, different kernels can be referred to as separate feature extractors. Two major hyperparameters that describe the convolution process are the size and number of kernels.

Figure 2.8 shows an example of convolution operation with a kernel size of $3x3$, no padding, and a stride of 1.

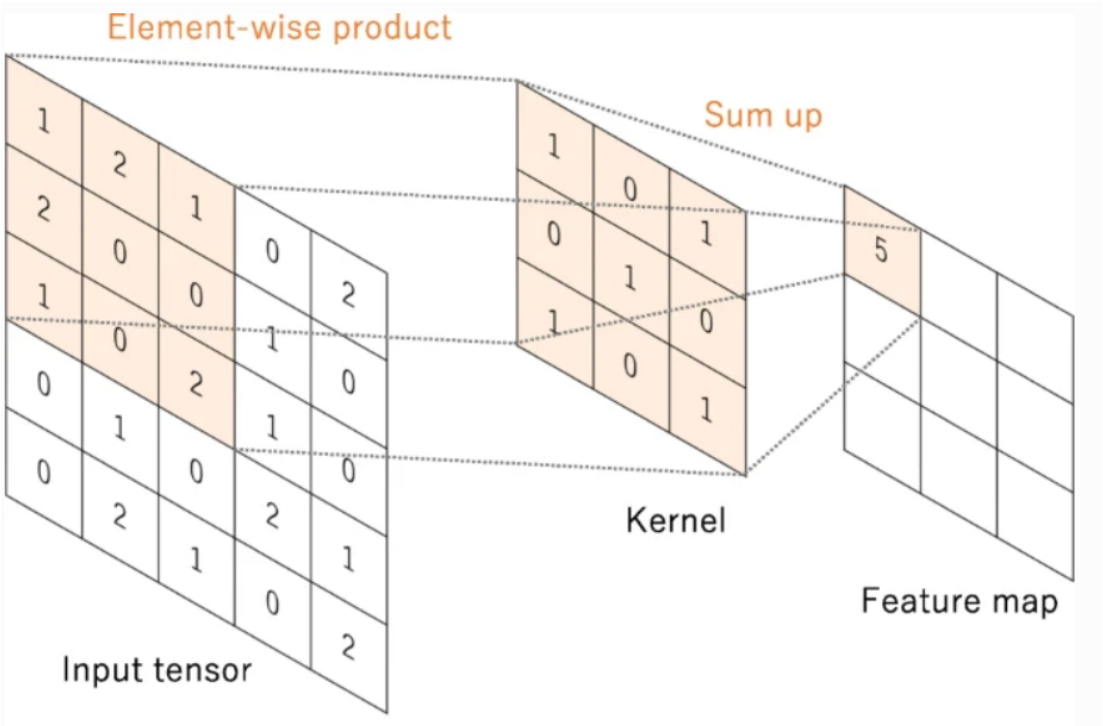


Figure 2.8: An example of convolution operation with a kernel size of $3x3$[16].

The convolution operation described in figure 2.8 does not allow the center of each kernel to overlap the outermost element of the input tensor. In addition, it reduces the height and width of the output feature map compared to the input tensor. Zero padding is a strategy for dealing with this problem that involves adding rows and columns of zeros on each side of the input tensor in order to fit the center of a kernel on the outermost area while keeping the same in-plane dimension via the convolution operation.

Figure 2.9 shows an example of convolution operation with a kernel size of

$3x3$, with padding, and a stride of 1.

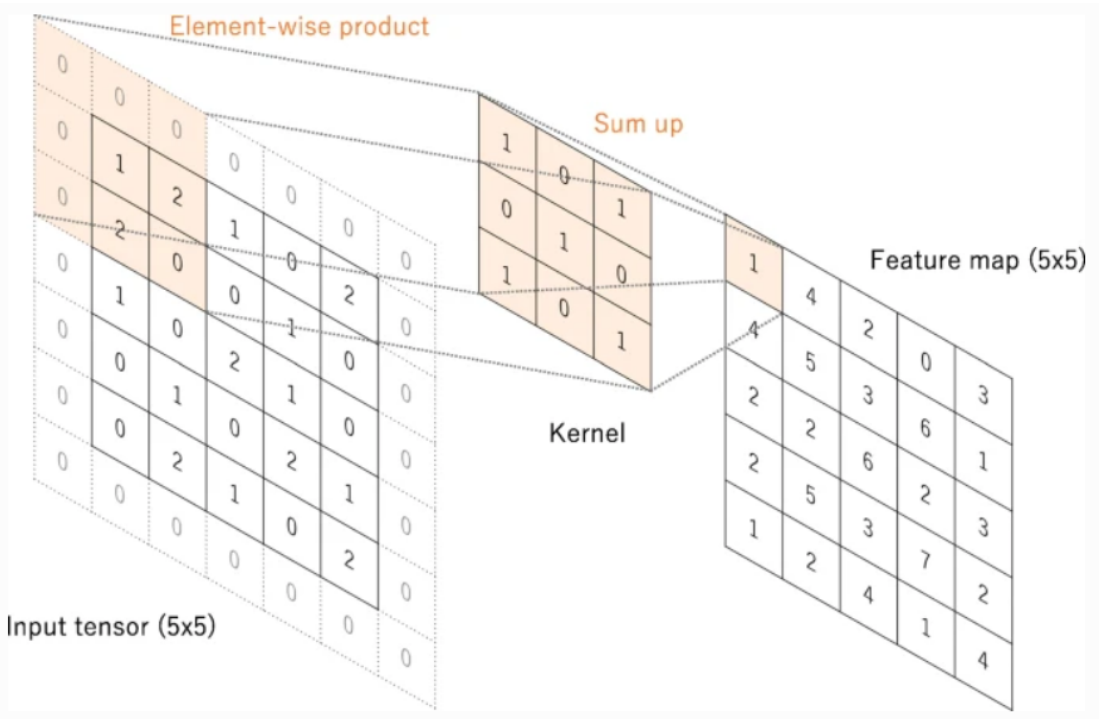


Figure 2.9: An example of convolution operation with a kernel size of $3x3$ with padding[16].

The distance between two successive kernel locations is called a stride. The most frequent stride is 1, though a stride greater than 1 is occasionally used to achieve feature map downsampling. A pooling process is an alternate method for downsampling.

Pooling Layer

The pooling layers reduces the number of parameters when the images are too large, which improves network performance. Spatial pooling, also known as subsampling or downsampling, reduces the dimensionality of each map while keeping the most important data.

Max-pooling and average-pooling are two common pooling filters. In max-pooling, a filter of size $[m \times m]$ slides over the input and takes the largest element from the rectified feature map. However, in average-pooing, the filter takes the average of elements from the the rectified feature map.

Figure 2.10 provides an example of max-pooing operation. In fact, a $[2 \times 2]$-filter is shifted over the 2-dimensional input with a stride of 2. The resulting output size is a quarter of the input size.

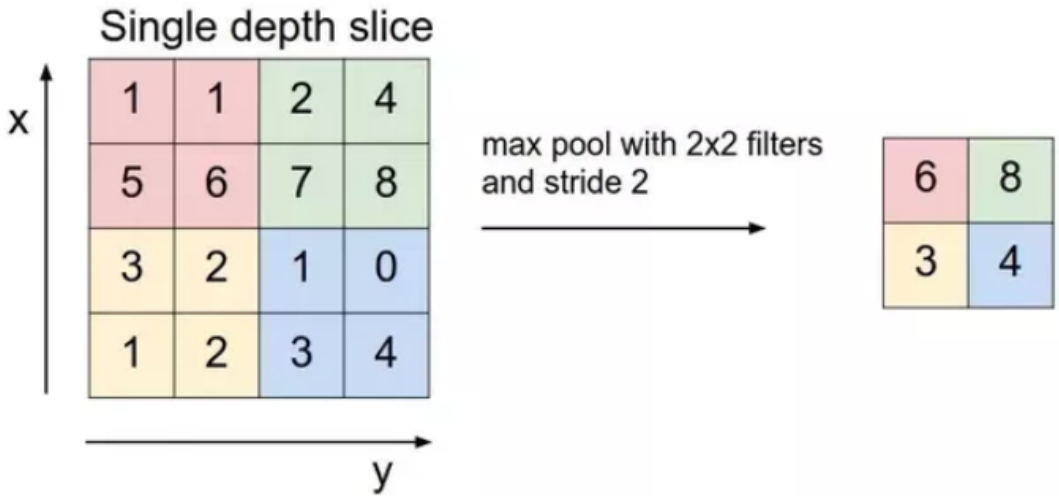


Figure 2.10: Example of max-pooling filter[15].

2.6.3 Transfer Learning

To obtain excellent performance using convolutional neural networks, we need a large amount of data and a lot of computer power. In fact, the more layers there are in the architecture, the more parameters there are to optimize. This is why computer manufacturers have created graphics processors (GPUs) that are efficient and capable of training deep neural networks quickly by parallelizing calculations.

However, apopular CNN architectures often have dozens of layers, hundreds of thousands of settings, and use hundreds of thousands of learning data. So even with GPUs, the training phase can last several days.

In order to solve these problems (calculation time and volume of data required), we use transfer learning which consists in using the knowledge acquired by a neural network during the resolution of a specific problem to solve another more or less similar.

Figure 2.11 shows the performance parameters-accuracy curve for representative ConvNets trained on Imagenet which is a very large collection of human annotated images designed by academics for developing computer vision algorithms.

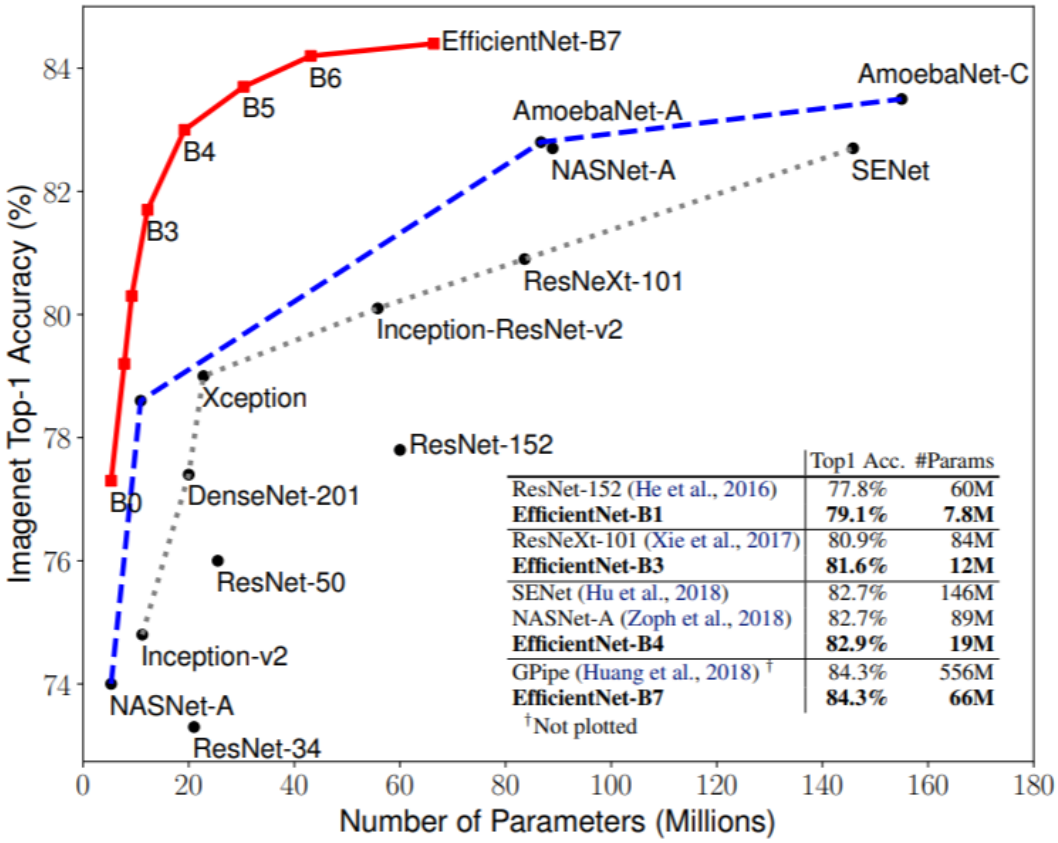


Figure 2.11: the performance parameters-accuracy curve for representative ConvNets trained on Imagenet[17].

ResNet-50

ResNet (Residual Network) is a pretrained Deep Learning model that is most frequently used to analyze visual imagery.

Identity connections are what distinguishes a residual network. In fact, identity connection routes the input directly to the end of each residual block, as shown in figure 2.12 with the curved arrow.

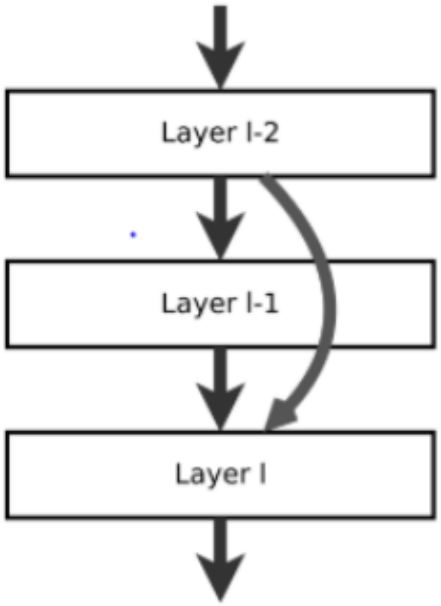


Figure 2.12: Residual block[15].

ResNet-50 is a 50-layer neural network that was trained on ImageNet. In addition, the model has over 23 million trainable parameters, indicating a deep architecture that performs well in image recognition. There are five stages in the ResNet-50 model, each with a residual block. Each residual block has three layers, each with convolutions of 1×1 and 3×3 and each layer in a feeds into the next layer and directly into the layers about 2–3 hops away, which is known as identity connection[4].

There are many other variants of ResNet architecture such as ResNet-18, ResNet-34, ResNet-101, ResNet-110, ResNet-152 etc.

2.6.4 Example of application field: Semantic Segmentation

Semantic segmentation is one of the most challenging research topic in computer vision today. With the rise in popularity of deep learning in recent years, many semantic segmentation problems are being addressed with deep architectures, most commonly Convolutional Neural Nets, which outperform other approaches in terms of accuracy and efficiency by a wide margin.

The aim of the image semantic segmentation challenge is to classify each pixel of an image (or a group of them) into a category, with each category corresponding to an entity or a part of the image.

UNET

U-net's architecture is composed of a contracting path to capture context and a symmetric expanding path that enables precise localization. This network can outperform the traditional methods (such as sliding-window convolutional network) by training end-to-end from few images[14].

The network architecture is illustrated in figure 2.13.

The left part in the architecture represents the contracting path however the right part refers to the expansive path.

The contracting path is composed of repeated application of two 3×3 convolutions, each followed by a rectified linear unit (ReLU) and a 2×2 max pooling operation with stride equal to 2 for downsampling. In each step of downsampling, we double the number of feature channels.

In the expansive path, every step is composed of an upsampling operation of the feature map followed by a 2×2 "up-convolution" that halves the number of feature channels. In addition, we use a concatenation with the correspondingly cropped feature map from the contracting path, and two 3×3 convolutions, each followed by a ReLU.

The final layer is a 1×1 convolution which is used to map each 64-component feature vector to the desired number of classes[14].

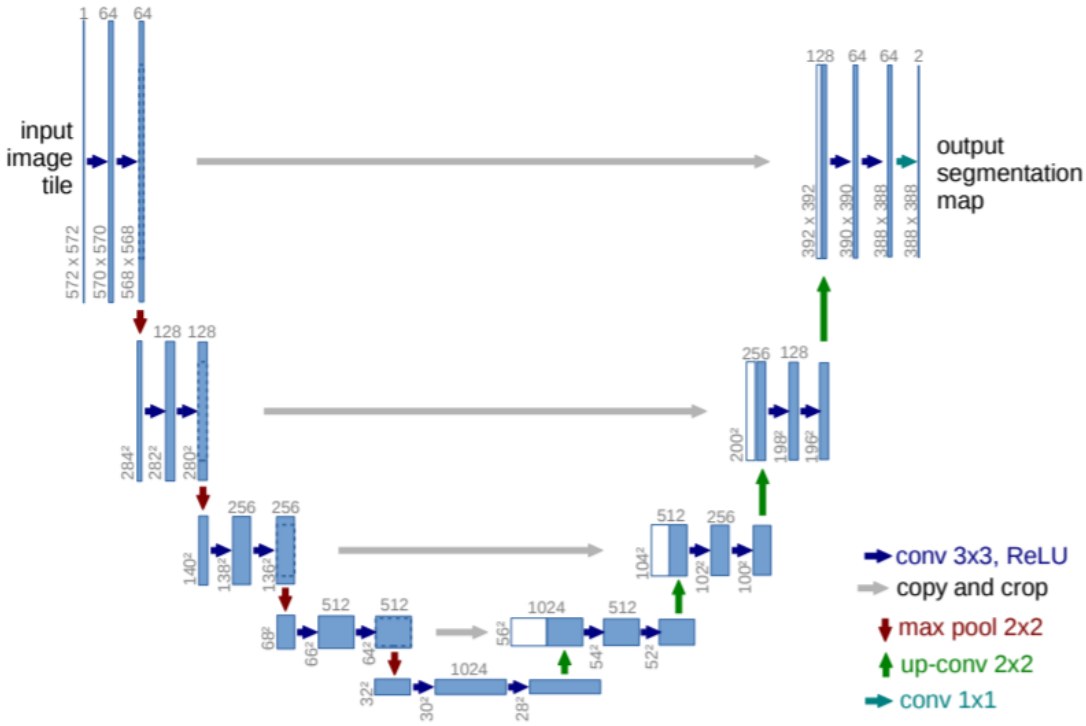


Figure 2.13: U-net architecture (example for 32×32 pixels in the lowest resolution). Each blue box corresponds to a multi-channel feature map. The number of channels is denoted on top of the box. The x-y-size is provided at the lower left edge of the box. White boxes represent copied feature maps. The arrows denote the different operations[14].

2.7 Conclusion

In this chapter, some basic notions necessary for the realization of the project were defined. We presented the concepts of PCA, One Class SVM, Gradient Boosting, Canny Edge Detection and Deep Learning as well as Convolutional Neural Network.

Chapter 3
Data and Problem Modeling

3.1 Introduction

This chapter goes over the planned work for this project in great detail. We begin by explaining the data. Following that, we refine our methods for estimating the fish's fork length, sex and weight. Finally, we go through the technology and programming languages that were used to complete the project.

3.2 Data

In this project, we use essentially images to build architectures and approaches for predicting the fork length, the sex and the weight of the fish.

3.2.1 Images

The dataset is composed of about 3000 images and one csv file containing the labels for each image.

Figure 3.1 represents an image example from the dataset.

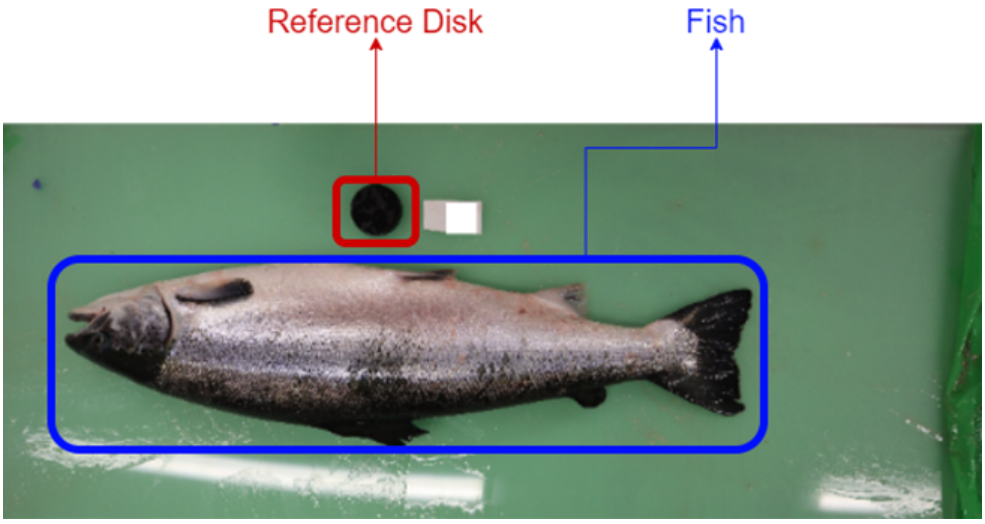


Figure 3.1: An image example from the dataset.

A fish and a reference disk are included in each image in the dataset. The reference disk has a purpose to calculate the length-to-pixel ratio. However, there are some images that were captured improperly and do not contain any fish.
In addition, we have image metadata which is text information embedded into the image file. Image metadata includes details relevant to the image itself as well as information about its production.

Table 3.1 shows a subsample of the metadata.

ID	Image Name	Shutter Speed	Aperture Value	ISO Speed	Focal Length
27	IMG_0120	6.625	3.625	125	15.0
28	IMG_1533	6.625	5.625	640	17.0
29	IMG_0219	6.625	3.625	160	15.0
...	...	...	...	...	...

Table 3.1: Subsample of the metadata

Those features logically, would not help in the prediction of weight, fork length or sex. As a result we can ignore them.

3.2.2 Targets

The fork length, the sex and the weight of the fish are the targets. They are stored in a csv file, with each row referring to a specific image.

Table 3.2 shows a subsample of the targets dataframe.

ID	Image Name	Sex	Weight	Fork Length
46	IMG_0457	F	4.355	73.0
897	IMG_0787	M	5.635	81.0
...	...	...	...	...

Table 3.2: Subsample of the targets dataframe

3.3 Solution Architecture

The theoretical studies carried out in the previous chapter are present together to establish an architecture that meets the functional requirements of the solution. This section is, therefore, devoted to the presentation of the overall structure of the project.

Figure 3.2 represents the global architecture of this project.

First of all, an outlier detection algorithm is used to confirm the existence of a fish in an input image. The image would be ignored if it does not contain a fish. Otherwise, it is fed into the fork length prediction part. In this part, the image is used by the two following approaches:

- **Image processing approach**
  The fish and the reference disk in the image are detected in this approach using an object detection algorithm based on traditional computer vision techniques. The fork length can then be estimated using a formula. The formula is explained in detail in the image processing approach section.
- **Machine Learning approach**
  In this approach, instead of processing the entire image we use semantic segmentation to select only the region of the fish. The output image is then fed into a regression model to predict the fork length.

To predict the sex of the fish we use the segmented image from the previous part as input to a binary classification model.

After predicting the fork length and the sex we can predict the weight by using them as input features for a regression model.

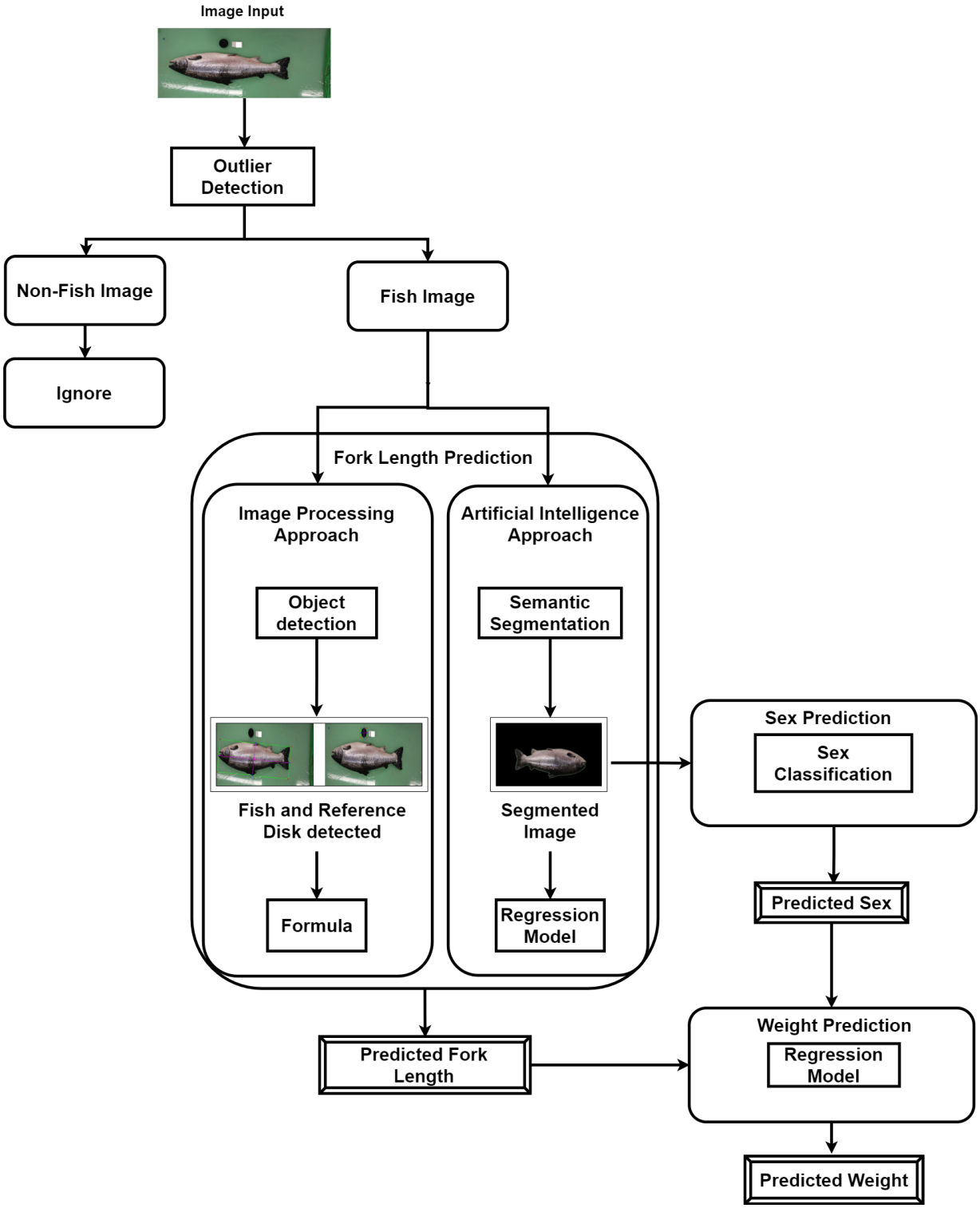


Figure 3.2: Global architecture.

3.4 Outlier detection

The first part of the global architecture is outlier detection. The objective of this part is to confirm the presence of a fish in the input image.

3.4.1 Reason behind using outlier detection

In this part we come across the task of building a binary classifier of the types A vs $\tilde{A}$, in which when the classifier is given a new data sample, it is able to predict whether the sample belongs to class A or is an outlier. In our case, given a new input image, we want to predict either this image is a fish or non-fish image. A few examples of non-fish images are shown in figure 3.3.

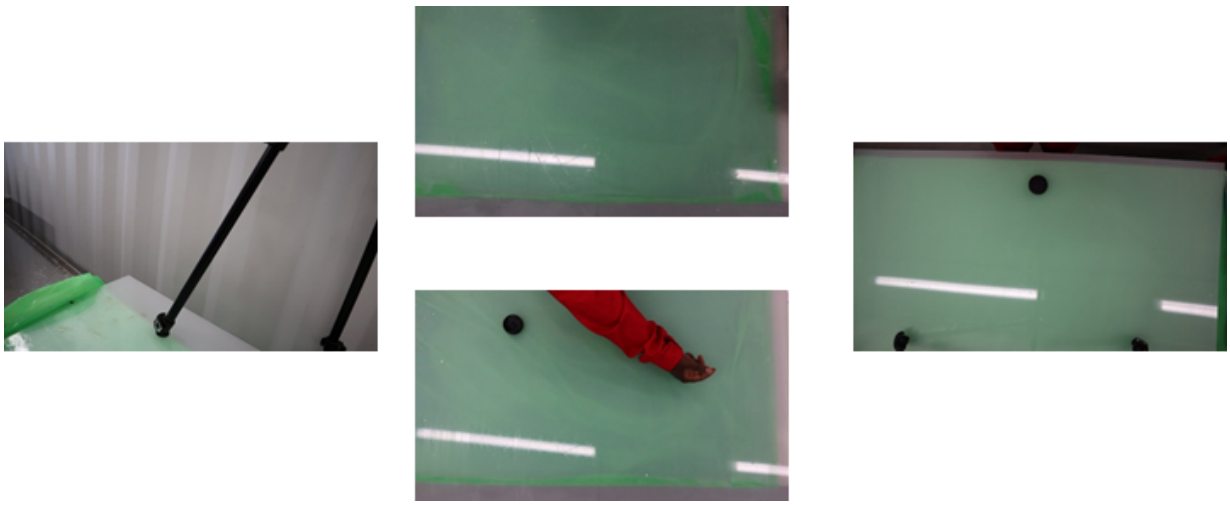


Figure 3.3: Examples of non-fish images.

The dataset is extremely imbalanced. In fact, only 0.57% of the images are non-fish image. Figure 3.4 shows a graphical display of the number of occurrences for fish and non-fish images.

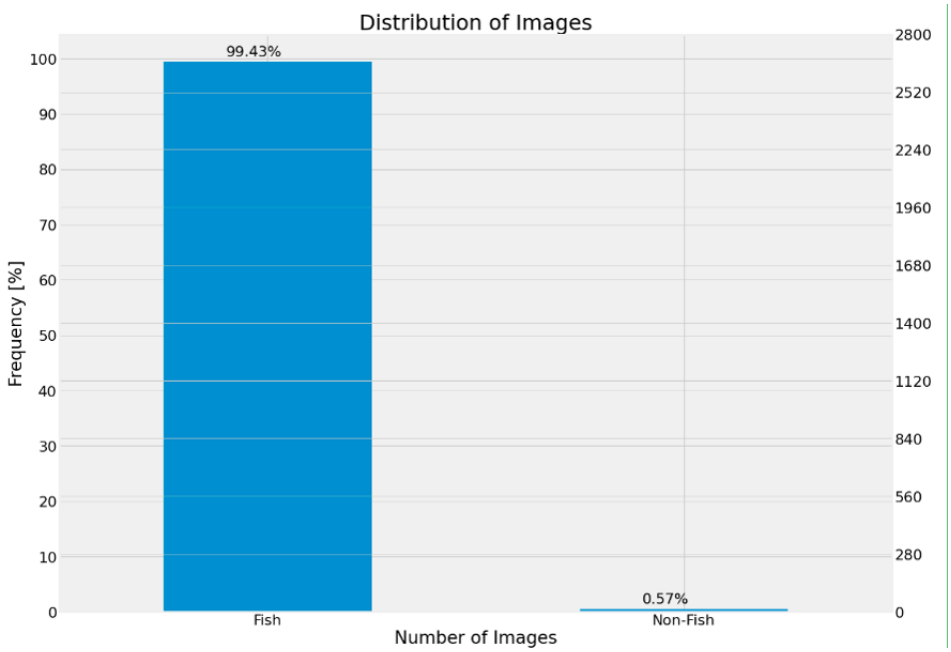


Figure 3.4: Number of occurrences for fish and non-fish images.

In a typical classification problem, we have almost an equal or a comparable number of positive and negative examples. So, generally when we want to build a classifier model we should have a balanced amount of positive and negative examples and we train our model on a good amount of positive as well as negative examples.

On the other hand, in outlier detection problem we have a significantly less amount of positive (anomalous) examples than the negative (non-anomalous) examples. The positive examples may be less than 5% or even 1% (obviously that is why they are anomalous). In such case, a classification algorithm cannot be trained well on positive examples.

Since the number of positive examples (non-fish images) is too small and the number of negative examples (fish images) is very large, traditional machine learning classifier would not perform well. This is the reason behind the use of an outlier detection algorithm in this part of the project.

3.4.2 Preprocessing and Feature Extraction

Resizing

Since neural networks and machine learning models receive inputs of the same size, all images need to be resized to a fixed size before inputting them to a specific model.

Feature Extraction

Feature extraction is a type of dimensionality reduction where a large number of pixels of the image are efficiently represented in such a way that interesting parts of the image are captured effectively. In computer vision, a feature is a measurable piece of data in the image which is unique to a specific part. It may be a distinct color in an image or a specific shape such as a line, edge, or an image segment.

In order to show the outlier detection model how to learn the characteristics of a fish image, we need to transform the input image into a feature vector which is a collection of features extracted from it.

Traditional feature extractors such as Harris Corner Detection, Scale-Invariant Feature Transform (SIFT) and Speeded-Up Robust Features (SURF) can be replaced by a convolutional neural network (CNN) because CNN's have a strong ability to capture the spatial and temporal dependencies in an image

through the application of relevant filters. Therefore, CNN can extract features that describe image in much more detail and efficient way. This allows us to simply use features returned from the deep CNNs and build our outlier detection model.

Figure 3.5 represents the process of creating a feature vector representative to the image.

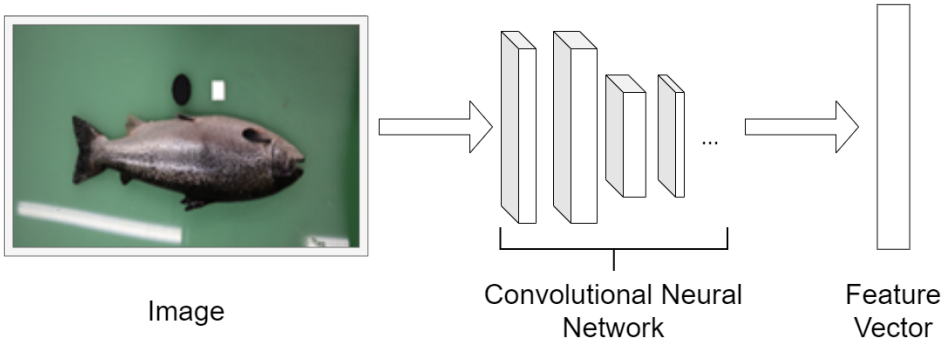


Figure 3.5: Transforming an image to a feature vector.

The availability of pre-trained CNNs on ImageNet data, with over 1,000 categories and more than 14 million images, has made image categorization much more simple since a pre-trained CNN will generally return features which are sufficiently satisfactory to train a light-weight model using them.

For the purpose of this project, we use ensemble averaging on multiple pre-trained CNN architectures on imagenet. In fact, we average the output feature vector from **ResNet-50**, **ResNet-101** and **ResNet-152**. The reason behind using ensemble averaging is its capacity to reduce the variance of predictions and reduce generalization error by combining the predictions from multiple neural network models and average them.

Scaling

We Apply standard scaler on the obtained features to give equal weights to all features since in PCA (next step preprocessing step) features with high variances or wide ranges, get more than those with low variance, and consequently, they end up dominating the first principal components. The equation (3.1) is used to perform data normalization.

$$X_{norm} = \frac{X - \nu}{\sigma}, \tag{3.1}$$

where:
ν: is the mean.
σ: is the standard deviation.

Dimensionality Reduction using PCA

Since the output feature vector has 2048 features, we should reduce the dimensionality of extracted features for quicker training time and reduce complexity in the training of outlier detection model. To do so, we use PCA which is a dimensionality-reduction method that is often used to reduce the dimensionality of large data sets, by transforming a large set of variables into a smaller one that still contains most of the information in the large set.

3.4.3 One Class SVM

We use One Class SVM as a machine learning model to perform the outlier detection. In fact, One Class SVM is an unsupervised algorithm that learns a decision function for outlier detection and classifies new data as similar or different to the training set. One Class SVM trains only on normal samples (fish images) and try to predict a new input image as normal or outlier sample.

3.5 Fork Length prediction

The objective of this section is to explain the two approaches used to predict the fork length of a fish from an image.

3.5.1 Target exploration

Fork length is the length of a fish measured from the tip of the snout to the end of the middle caudal fin rays.

Figure 3.6 represents the distance of fork length of a fish.

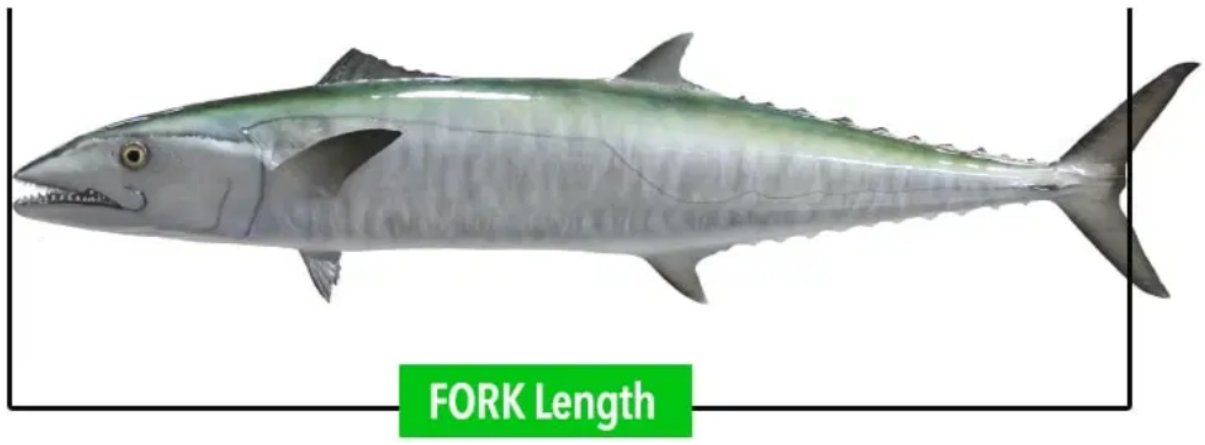


Figure 3.6: Distance of fork length.

In the dataset, as shown in figure 3.7, the fork length follows a bell shape curve. The maximum value is 89 and the minimum value is 46. About 99% of fork length's values are between 60 and 89.

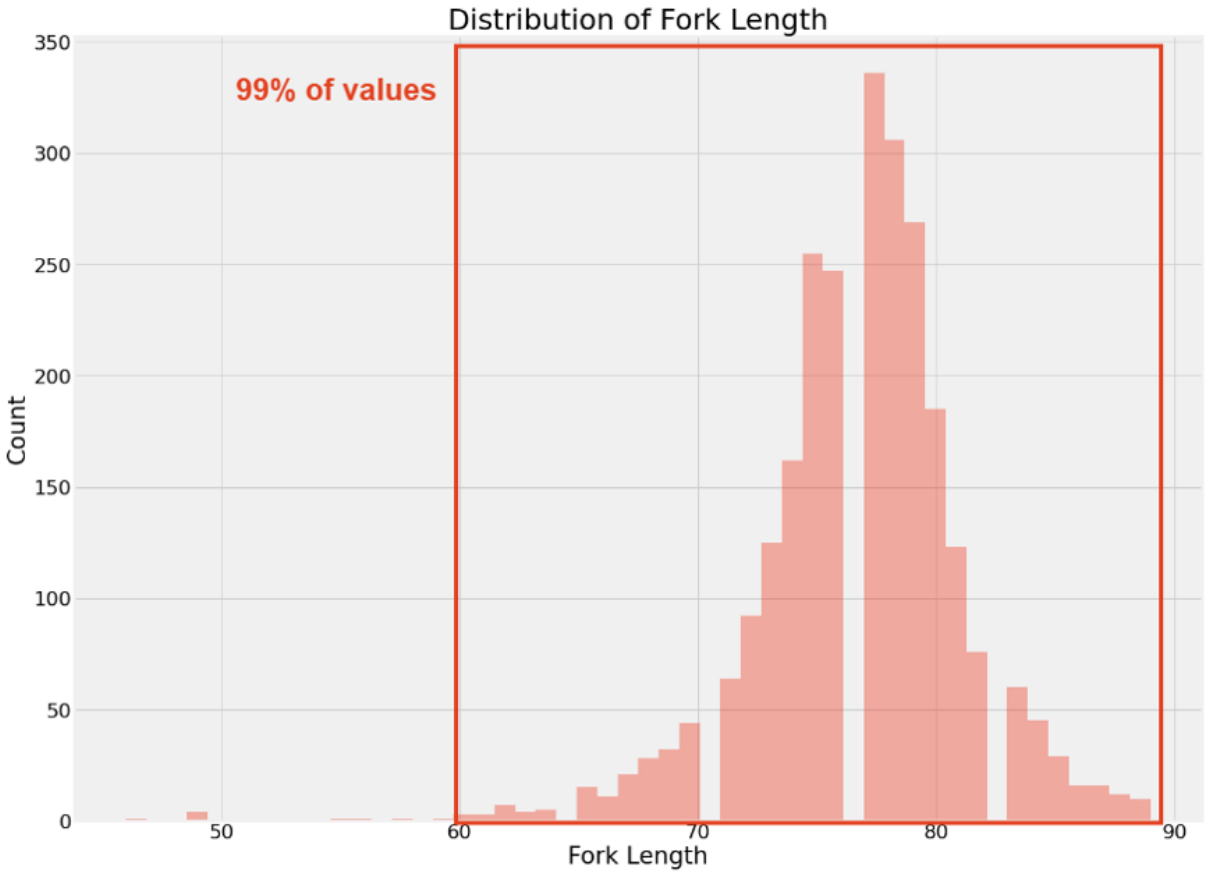


Figure 3.7: Distribution of the fork length.

3.5.2 Image Processing approach

The objective of this section is to predict the fork length of the fish using traditional computer vision techniques.

Architecture

Figure 3.8 shows the architecture of the Image Processing approach.

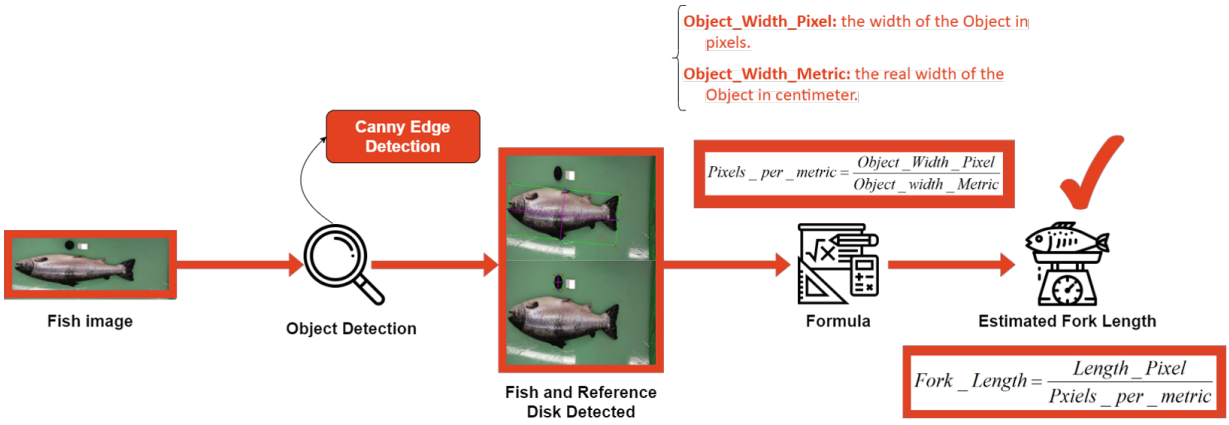


Figure 3.8: Image processing approach.

In this approach, using object detection, we first detect the fish and the reference disk in the image. Object detection is usually achieved by edge detection. That's why we use canny edge detection to detect the objects. Knowing the real width of the reference disk in centimeters we can use the formula in equation (3.3) to calculate the *pixels_per_metric*.

$$pixels_per_metric = \frac{Disk_Width_Pixel}{Disk_Width_Metric}, \tag{3.2}$$

where:
pixels_per_metric - number of pixels per 1 centimeter.
Disk_Width_Pixel - width of the reference disk in pixels.
Disk_Width_Metric - real width of the reference disk in the given metric (centimeter).

Finally, we can estimate the fork length of the fish using equation (3.3).

$$Fork_Length = \frac{Length_Pixel}{pixels_per_metric}, \tag{3.3}$$

where:
Fork_Length - estimated fork length in centimeters.
Length_Pixel - length of the fish in pixels.

Preprocessing steps

To get a clean input without unwanted objects for the Canny Edge Detection algorithm. We first need to do some preprocessing steps:

- **Resizing**
  In order to process faster and easier the image, image resizing is needed.

- **Removing the background**
  The background's pixels have a specific property. In fact, as shown in figure 3.9, pixels with a difference between their green and red components greater than 14 are background pixels. However, pixels inside the fish, the reference disk or the reflected light of the tube have almost an equal value for their three components red, green and blue. To eliminate the background, we use a filter that converts pixels with a difference greater than 14 between their green and red components to white.

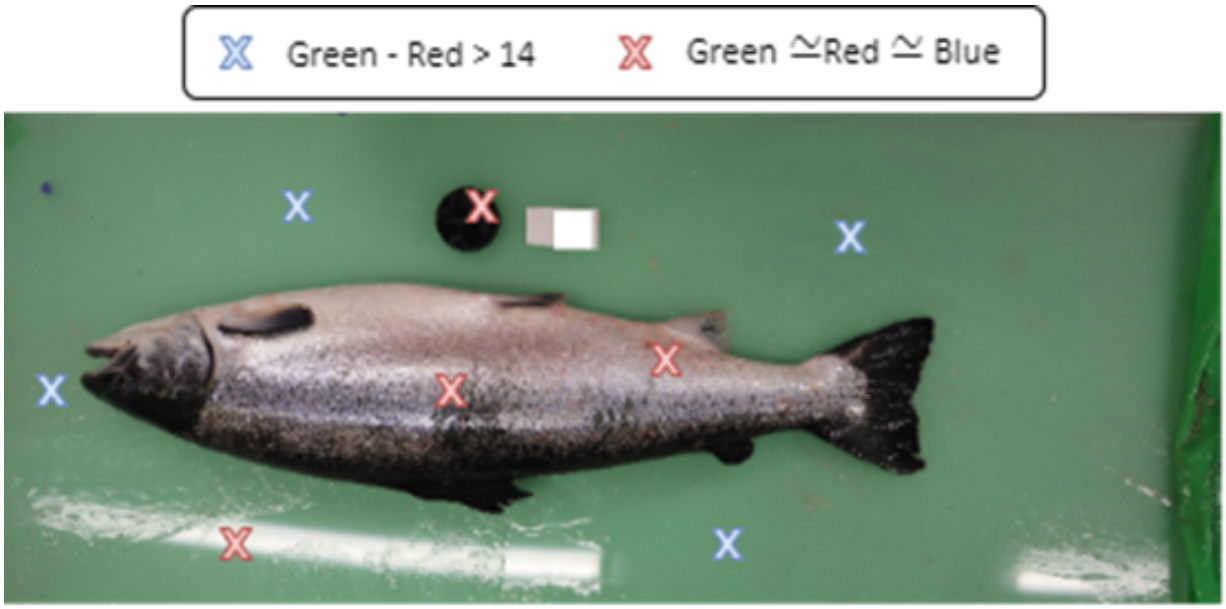


Figure 3.9: Difference between background and not background pixels.

Figure 3.10 illustrates the output image after applying resizing and removing background steps.

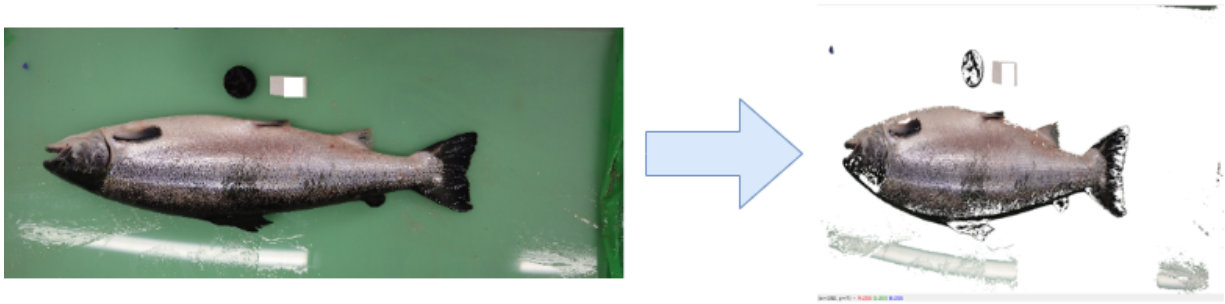


Figure 3.10: output image after applying resizing and removing the background steps.

- **Transforming to grayscale and eliminating noise**
  We first transform the image to grayscale. Then, in order to remove

the pixels outside the fish and reference disk, we apply thresholding. In fact, pixels with a gray value greater than 180 are unlikely to be fish or reference disk pixels. We use then a filter that converts those pixels to white pixels (gray value equal to 255).

Figure 3.11 represents the output image after applying thresholding.



Figure 3.11: output image after applying thresholding.

Canny Edge detection

Edge detection is a fundamental tool aimed at identifying transition in image where brightness changes sharply.

Canny edge detection is an operator that detects edges in a given image using multi stage algorithm. Smoothening, locating gradients, non-maximum suppression and hysteresis thresholding are the four key stages in this multi-stage algorithm.

Figure 3.12 shows the detected edges using canny edge detection.



Figure 3.12: Detected edges using Canny Edge Detection algorithm.

Dilation and Erosion

Morphological Filters have two basic functions: Erosion and Dilation. In order to close gaps in between edges in the edge map and connect small bright cracks, we use a closing filter which is a morphological filter where dilation is follwed by erosion. This filter starts with dilation operation where dark regions that are smaller than the structuring element are removed. The erosion operation that follows ensures that dark regions that are larger than the structuring element retain their original size.

Figure 3.13 shows the output image after applying closing filter.

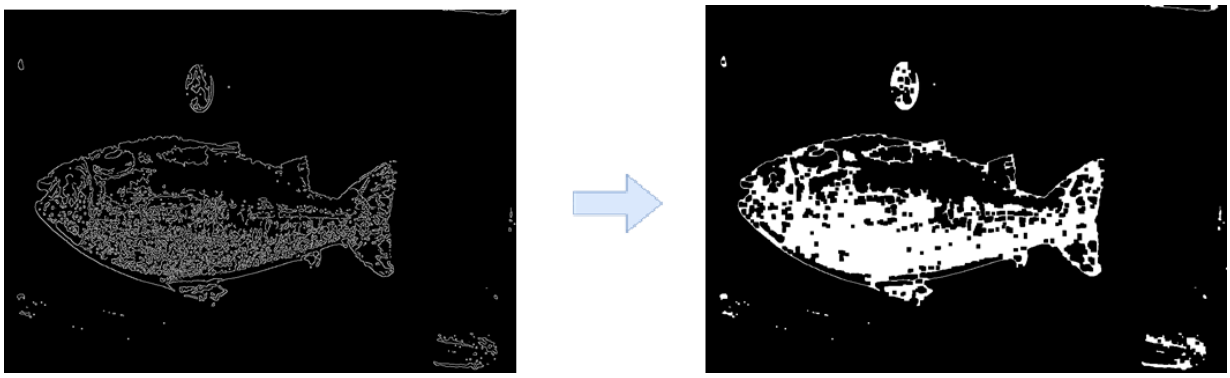


Figure 3.13: Output image after applying closing filter.

Find contours

In this step, we aim to find contours that correspond to either the fish or the reference disk.

For pixels where we detect change in the image color (from black to white or inversely), we mark them as contour. We start looping over each of the individual contours. If the contour is not sufficiently large, we discard the region, presuming it to be noise left over from the edge detection process. Provided that the contour region is large enough, we compute the rotated bounding box that covers the entire contour.

The figure 3.14 represents the detected objects in the image.

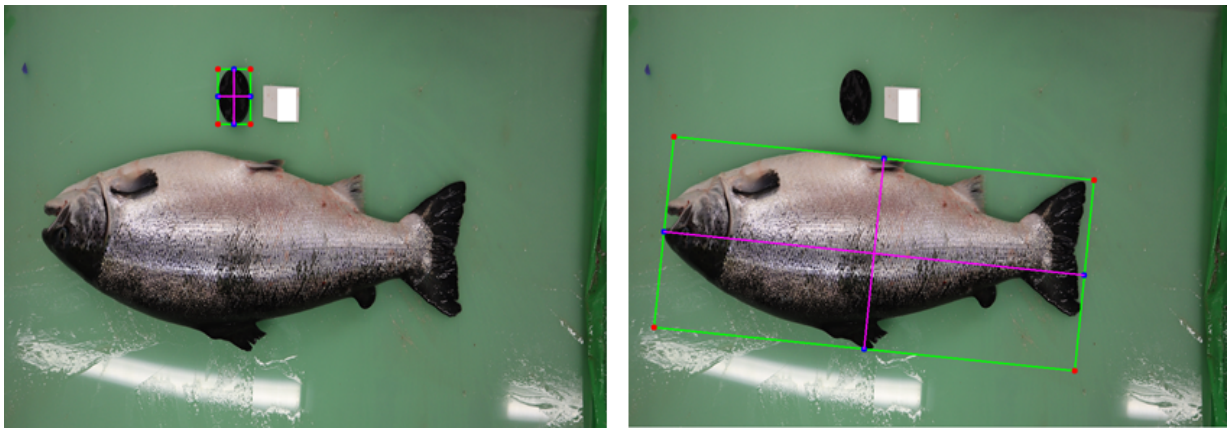


Figure 3.14: Detected objects in the image.

3.5.3 Machine Learning Approach

The objective of this section is to predict the fork length of the fish using computer vision and machine learning techniques.

Architecture

Figure 3.15 shows the architecture of the Machine Learning approach.

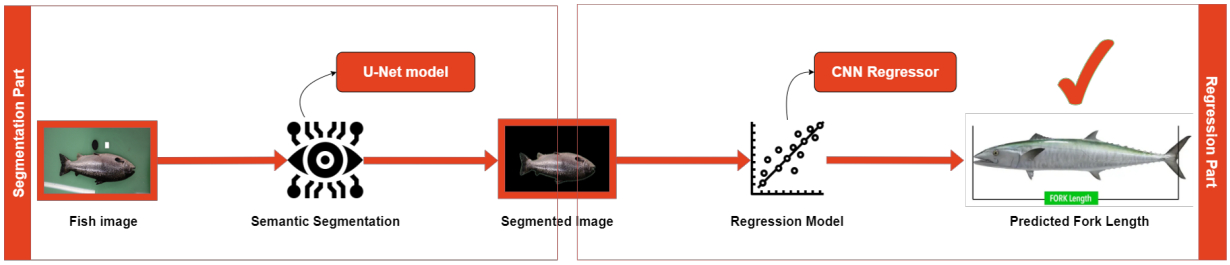


Figure 3.15: Machine Learning approach.

In this approach, we have two parts:

- **Segmentation Part**
  We detect the pixels of the fish using semantic segmentation model which is U-Net. We get then as output a segmented image containing only pixels of the fish and other pixels are set to black.
- **Regression Part**
  We use the segmented image from the segmentation part as input to a regression model to predict the fork length.

Semantic segmentation

Semantic segmentation involves converting an image into a collection of regions of pixels that are represented by a mask. By dividing an image into segments, we can process only the important segments of the image instead of processing the entire image. Since our aim at the end is to predict the fork length of the fish, we use semantic segmentation to extract only the fish from the input image.

In our case, we want to assign to each pixel of the image a label depending if it belongs to the fish or not. Since the data is unlabeled, we first need to create a mask for every image. Each pixel in the mask is given one of two categories:

- **White value:** Pixel belonging to the fish.
- **Black value:** Pixel not belonging to the fish.

There are many existing tools to create labelled masks for images, including Labelme. To create a mask using Labelme, we draw a polygon that covers the entire fish by using points to delimit the region of the fish. Then we generate a binary image where we assign the white color for pixels inside the polygon and black color for pixels outside it.

Figure 3.16 represents the created mask using Labelme tool.

Images and masks are then resized, normalized and fed to U-Net model which is convolutional network architecture for fast and precise segmentation of images.

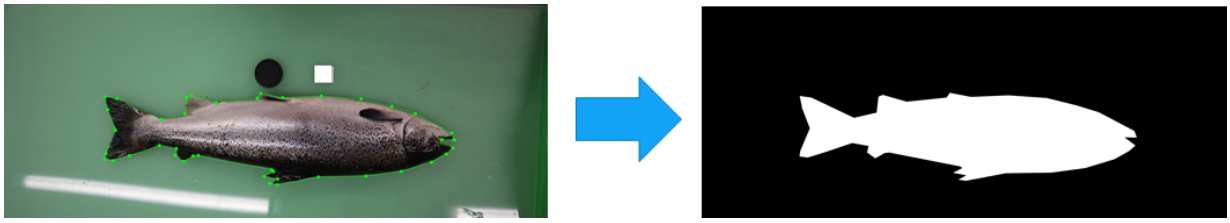


Figure 3.16: Created mask using Labelme tool.

Regression

After segmenting the image, we use it as input to a regression model to predict the fork length. This model is a pretrained **ResNet18** on imagenet. Its a convolutional neural network that is 18 layers deep pretrained on more than a million images from imagenet dataset. The segmented images used to train this model are the refined predictions from the semantic segmentation model.

3.6 Sex Prediction

The objective of this section is to explain the methodology adopted to predict the Sex of the fish using the segmented image from the previous section.

3.6.1 Target exploration

The dataset contains two categories for the target Sex:

- Male (M): 2294 samples about 87.3% of total samples.
- Female (F): 334 samples about 12.7% of total samples.

Figure 3.17 shows the number of occurrences for Male and Female fishes.

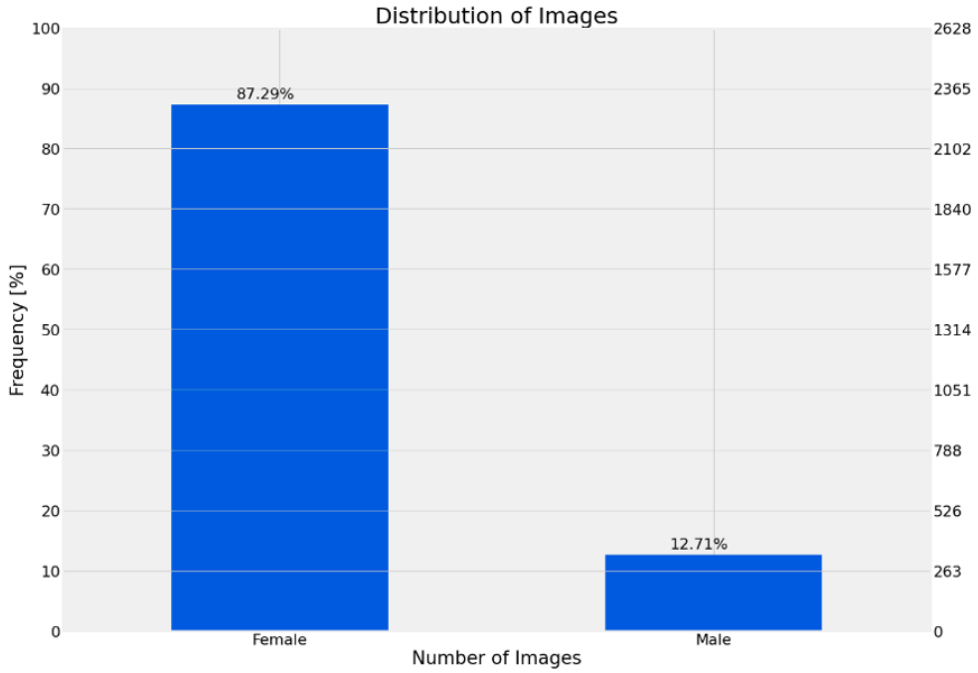


Figure 3.17: Number of occurrences for Male and Female fishes.

The dataset is imbalanced, we should deal with this problem in the preprocessing steps.

3.6.2 Preprocessing and Training

Preprocessing Steps

The preprocessing steps are:

- **Resizing**
  Since neural networks receive inputs of the same size, all images need to be resized to a fixed size before inputting them to the CNN model.

- **Normalization**
  We Apply standard scaler to the images to ensure that each pixel has a similar data distribution.
- **Technique used to deal with imbalanced dataset**
  To deal with imbalanced dataset we use **random oversampling**. It consists in over-sizing the minority class by randomly duplicating observations.

  Figure 3.18 shows a visual explanation of under-sampling and over-sampling.

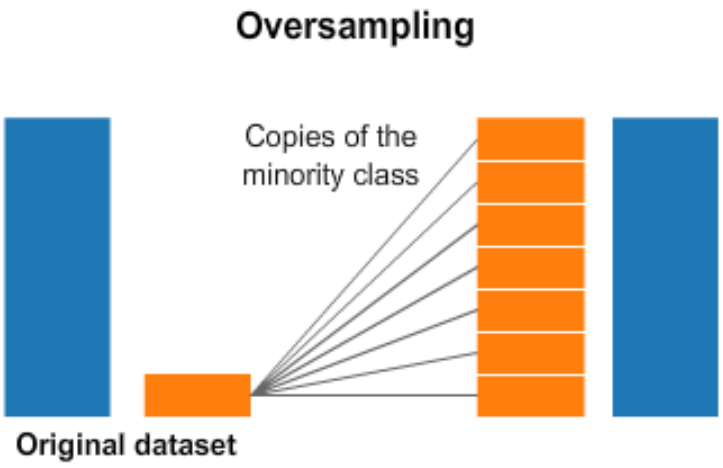


Figure 3.18: Visual explanation of over-sampling.

Training

We use pretrained CNN architectures on imagenet such as **ResNet18**, **ResNet50** and **ResNet152** which are variations of ResNet architecture.

Since the dataset has low similarity to the imagenet dataset, it is significant to retrain the higher layers of the different architectures. However, initial layers are kept pretrained and the weights for those layers are frozen. We also add last layer to match the classes in our dataset.

3.7 Weight Prediction

The objective of this section is to explain the methodology adopted to predict the weight of the fish using the predicted fork length and the segmented image from the previous section.

3.7.1 Target Exploration

In the dataset, the weight follows a bell shape curve. About 98% of weight's values are between 2 and 8. The maximum value is 9.27 and the minimum value is 0.96.

Figure 3.19 shows the distribution of the weight.

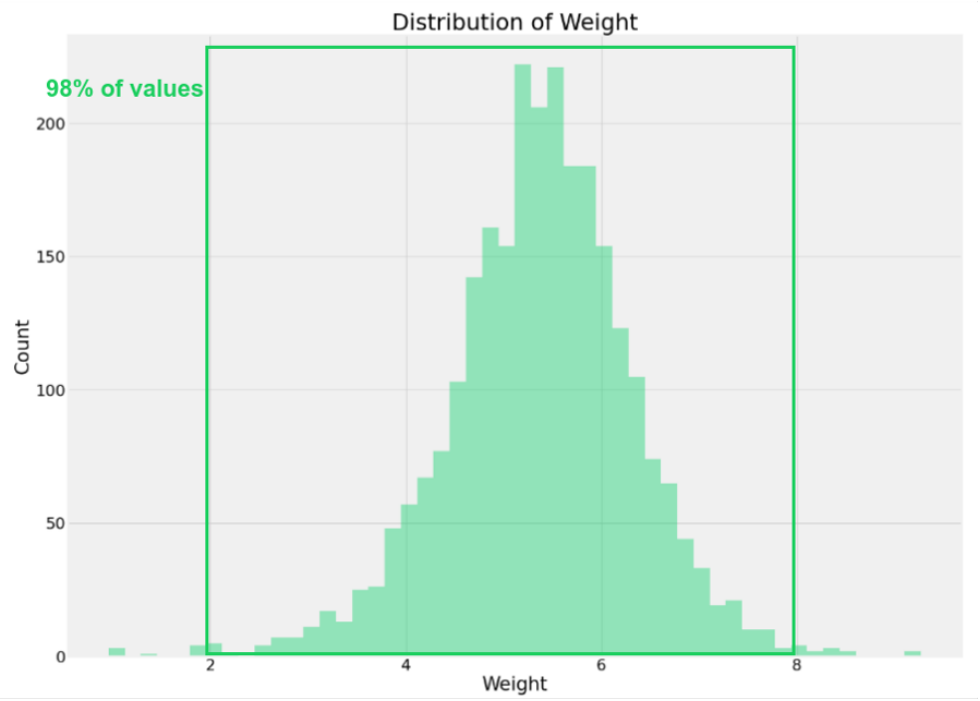


Figure 3.19: Distribution of the weight.

3.7.2 Architecture

Figure 3.20 shows the architecture used to predict the weight.

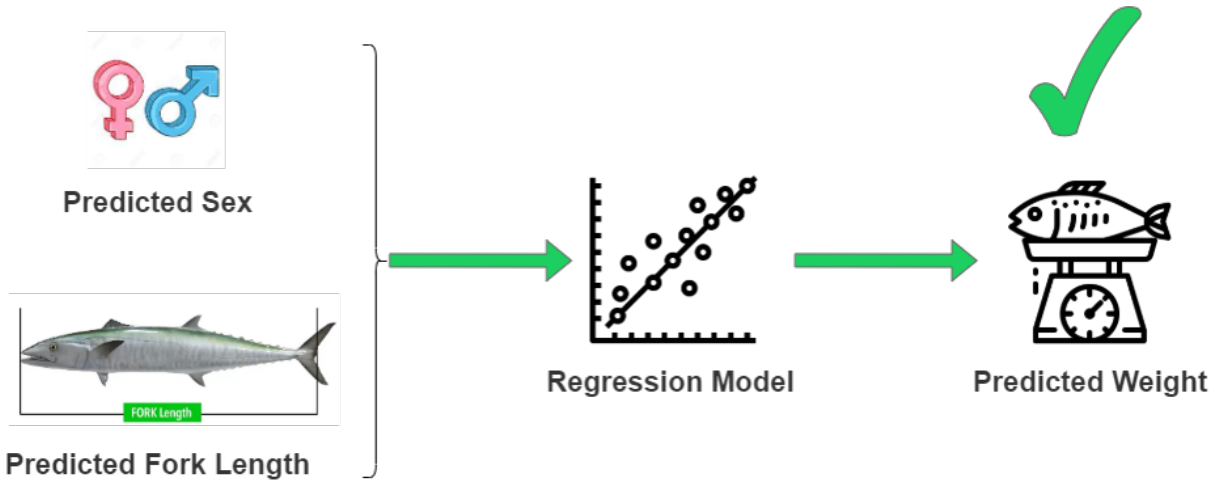


Figure 3.20: Architecture for weight prediction.

We simply use the predicted sex and the predicted fork length as inputs to a regression model to predict the weight.

3.7.3 Correlation With Target

From the plot 3.21 we can assume that the longer the fish is the bigger the weight. For a given fork length, female fishes are usually heavier than male fishes. In addition, male fishes are usually longer than female fishes. As a result, we can conclude that also sex correlates positively with the Weight. Those two features should help the regression model to give good prediction for the weight.

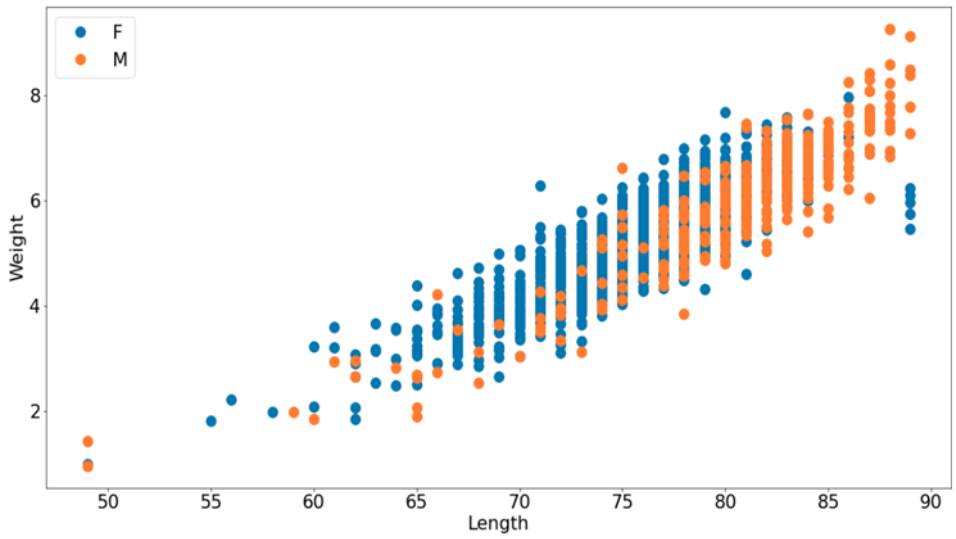


Figure 3.21: Correlation between the fork length, the sex and the weight.

3.7.4 Training

We use Gradient Boosting models such as XGBoost and LightGBM to perform regression. The two main reasons to use Gradient Boosting models are performance and execution speed. In fact, they are an ensemble learning algorithms, which combine the predictions of multiple base learners (usually, binary decision trees) to generate one overall prediction for each input. This allows them to learn more complex relationships between the features and target in the training set. Moreover, what makes, for example, XGBoost fast is its capacity to do parallel computation on a single machine.

3.8 Implementation Details

In this section, we detail the specifications of the machines as well as different software and technologies used in this work.

3.8.1 Hardware Environment

All necessary scripts related to this work have been deployed on a machine with the following characteristics:

Operating System	Ubuntu 20.04.2 LTS
CPU	Intel(R) Core(TM) i7-6600U CPU @ 3.4GHz
RAM	16 GB

Table 3.3: Hardware Characteristics

3.8.2 Software Environment

To implement the models that we have conceived, we used a wide range of technologies and software that we list here:

Python

Python is the world's most popular and fastest growing programming language. It comes with several very attractive machine learning libraries as well as powerful set of packages for analytical needs. This encouraged us to choose python over the other programming languages. Furthermore, Python has a very large community that provides support through forums.

Numpy

Numpy is the fundamental package for scientific computing with Python. It allows us to process arrays and perform operations related to linear algebra.

Pandas

Pandas is an open source library. We chose to use Pandas because it allows importing data from various file formats and provides various data manipulation and data analysis built in functions.

PyTorch

PyTorch is an open source machine learning library based on the Torch library, used for applications such as computer vision and natural language processing, primarily developed by Facebook's AI Research lab. We used PyTorch for various reasons. In fact, PyTorch supports using naive python code letting us write code in a more pythonic way. Moreover, PyTorch has great set of simple APIs and it is easy to debug PyTorch code.

OpenCV

OpenCV is a library of programming functions mainly aimed at real-time computer vision. Since this is a computer-vision based project, we used opencv because it is free open-source mainly focuses on image processing and provides built in functions to work with images.

Scikit-learn Package

Scikit-learn is one of the best-known libraries which provide solid implementation of machine learning algorithms. In addition, Scikit-learn has efficient tools for data analysis. We mainly used scikit-learn to implement machine learning models.

3.9 Conclusion

In this chapter, we described the data used during this work. Next, we presented the architectures and the models used to estimate the fork length, the weight and the sex of the fish. Finally, we have shown the software and hardware environments for the implementation and testing of this project.

Chapter 4
Experimental Results

4.1 Introduction

In this chapter, we are going to showcase the results attained after implementing the previsouly described architectures. To do so, we will first describe the error metrics used to evaluate these results.

4.2 Error Metrics

In this section, we present the error metrics used to evaluate our results.

4.2.1 Percentage Error

Percentage error calculates the percentage of the error relative to the actual values.

$$PE = 100 \times \frac{Predicted - Actual}{Actual} \tag{4.1}$$

Boundary conditions are:

- If Actual = 0 and Predcited = 0 then PE = 0%
- If Actual = 0 and Predicted > 0 then PE = 100%
- If Actual = 0 and Predicted < 0 then PE = -100%

4.2.2 Mean Absolute Percentage Error (MAPE)

The Mean Absolute Percentage Error (MAPE) is the average of the Absolute Percentage Error over a number of observations. Its formula is shown in equation (4.2).

$$MAPE = \frac{100}{n} \sum_{i=1}^{n} \frac{|Actual_i - Predicted_i|}{Actual_i} \tag{4.2}$$

Just as Mean Absolute Error (MAE) is the average magnitude of error produced by the model, the MAPE is how far the model's predictions are off from their corresponding outputs on average. Like MAE, MAPE also has a

clear interpretation since percentages are easier for people to conceptualize. The figure 4.1 shows a visual explanation of MAPE.

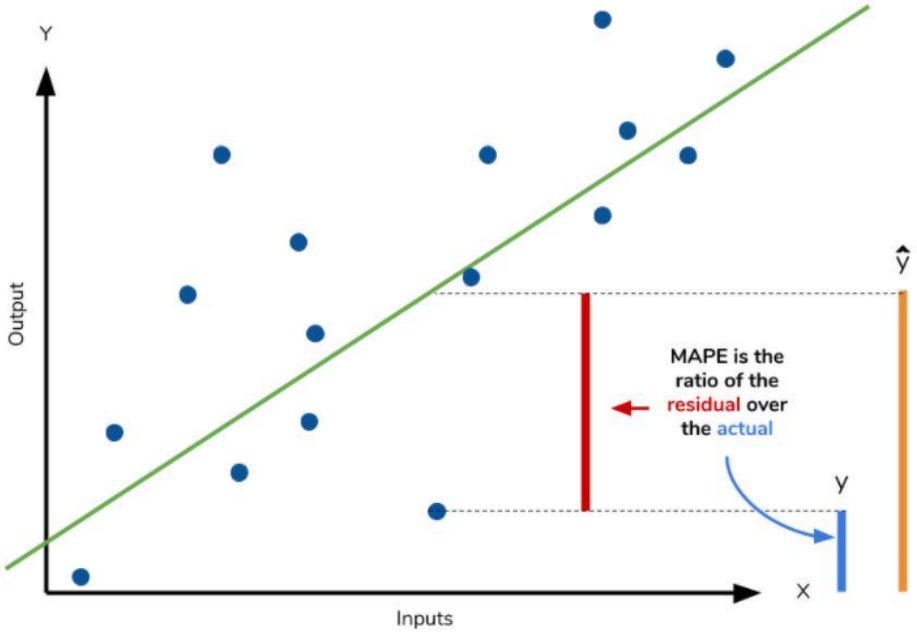


Figure 4.1: Visual explanation of MAPE.

However for all of its advantages, MAPE have some weaknesses. First of all, MAPE is undefined for data points where the value is 0. In addition, the MAPE can grow unexpectedly large if the actual values are exceptionally small themselves. Finally, the MAPE is biased towards predictions that are systematically less than the actual values themselves. That is to say, MAPE will be lower when the prediction is lower than the actual compared to a prediction that is higher by the same amount.

4.2.3 Accuracy and F1-score

Confusion Matrix

Confusion Matrix for binary classification is a 2×2 matrix with the structure represented in figure 4.2:

- The element number of true negatives is the number of rows classified by the model as 'Negative' and are actually 'Negative'.
- The element number of false negatives is the number of rows classified by the model as 'Negative' but are actually 'Positive'.
- The element number of true positives is the number of rows classified by the model as 'Positive' and are actually 'Positive'.

- The element number of false positives is the number of rows classified by the model as 'Positive' but are actually 'Negative'.

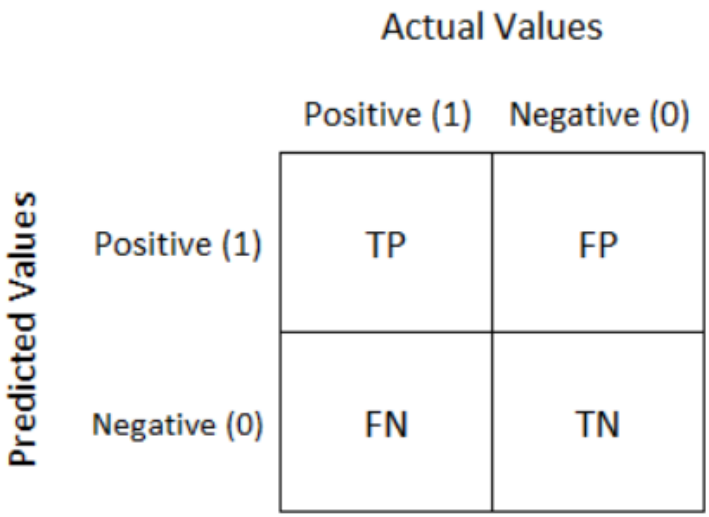


Figure 4.2: Confusion Matrix for binary classification.

Precision

It is implied as the measure of the correctly identified positive samples from all the predicted positive samples. Precision has the following definition:

$$Precision = \frac{True\ Positvie}{True\ Positive + False\ Positive} \tag{4.3}$$

Accuracy

Accuracy is one metric for evaluating classification models. Its the fraction of predictions our model predicted correctly. Accuracy has the following definition:

$$Accuracy = \frac{True\ Positvie + True\ Negative}{True\ Positive + False\ Positive + True\ Negative + False\ Negative} \tag{4.4}$$

Classification accuracy is the most-used metric for evaluating classification models. The reason for its wide use is because it is easy to calculate, easy to interpret, and is a single number to summarize the model's capability.

However, accuracy fails for imbalanced classification problems, where the distribution of examples in the training dataset across the classes is not equal.

In fact, achieving 90% classification accuracy, or even 99% classification accuracy, may be trivial on an imbalanced classification problem. This means that accuracy misleads us into thinking that a model has good or even excellent performance when it does not. In such case, we should use F1-score metric to evaluate the model.

Recall

It is the measure of the correctly identified positive samples from all the actual positive samples. Recall has the following definition:

$$Recall = \frac{True\ Positvie}{True\ Positive + False\ Negative} \tag{4.5}$$

F1-Score

F1-Score is the weighted average of Precision and Recall. F1-score is used in the case where we have skewed classes i.e one type of class examples more than the other type class examples. Mainly we consider a case where we have more negative examples than positive examples. F1-score has the following definition:

$$F1score = 2 \times \frac{Precision \times Recall}{Precision + Recall} \tag{4.6}$$

4.2.4 Dice Coefficient

Dice Coefficient is a metric used to evaluate a semantic segmentation model. Simply, the Dice Coefficient is 2× The Area of Overlap divided by the total number of pixels in both images (true mask and predicted mask).

Figure 4.3 shows an illustration of Dice Coefficient.

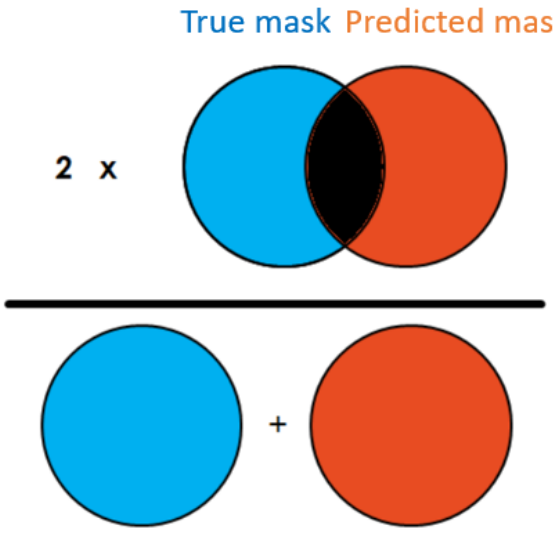


Figure 4.3: Illustration of Dice Coefficient.

Dice Coefficient ranges from 0 to 1, with 1 signifying the greatest similarity between predicted and truth.

4.3 Outlier Detection

4.3.1 settings

Dataset and Cross Validation

The dataset consists of 2816 images. Only 16 images are Non-fish images. We split our data into training, validation and testing. There are 2000 training images, 408 validation images and 408 testing images. The split of the dataset is shown in table 4.1.

	Training	**Validation**	**Testing**
Total images	2000	408	408
Fish images	2000	400	400
Non-Fish images	-	8	8

Table 4.1: Splits of the dataset.

We resize the images to $(124, 124, 3)$ and convert them from RGB to BGR, then each color channel is zero-centered with respect to the imageNet dataset, without scaling.

We apply average ensembling on predictions from those pretrained architectures: **ResNet-50**, **ResNet-101** and **ResNet-152**.

Each architecture takes as input preprocessed images and predict an array containing a 2048-feature vector associated for each image. The figure 4.4 represents an example of feature vector associated to a single image.

```
array([1.3194581 , 0.31340075, 0.89120495, ..., 1.84083   , 0.03462183,
       0.16842216], dtype=float32)
```

Figure 4.4: Example of extracted feature vector.

The output feature vector is then scaled using standardizing. We apply also PCA which is used to reduce the dimensionality of extracted feature allow for quicker training times.

The percentage of variance explained by the PCA representation reflect the percentage of information that this representation bring about the original structure. Higher is the percentage of variance, higher is the percentage of information and less is the information loss. The hyperparameter **n_components** of the PCA algorithm is chosen so that the percentage of variance is big enough. The value chosen was 448 which corresponds to a percentage of variance equal to 92%.

Choice of kernel

When a linear decision boundary cannot separate classes, One-Class SVM has the ability to project space through a non-linear function, lifting the data to a space with a higher dimension where a linear decision boundary does separate classes. As a result, we should adjust the kernel hyperparameter to obtain a better decision boundary.

We can tune the kernel parameter by visualizing the decision boundary for each possible value. However, we first need to decrease the dimensionality of the dataset to $2D$ so we can visualize the obtained decision boundary. Because One-Class SVM only trains on normal samples (not outliers), the produced decision boundary should cover as many data points as possible. A new data point outside of the decision boundary would therefore be classified as an outlier.

Kernel parameter has three possible values:

- **"linear":** using this parameter we can get a linear separable hyperplane (boundary). Figure 4.5 shows the decision boundary obtained for a linear kernel.

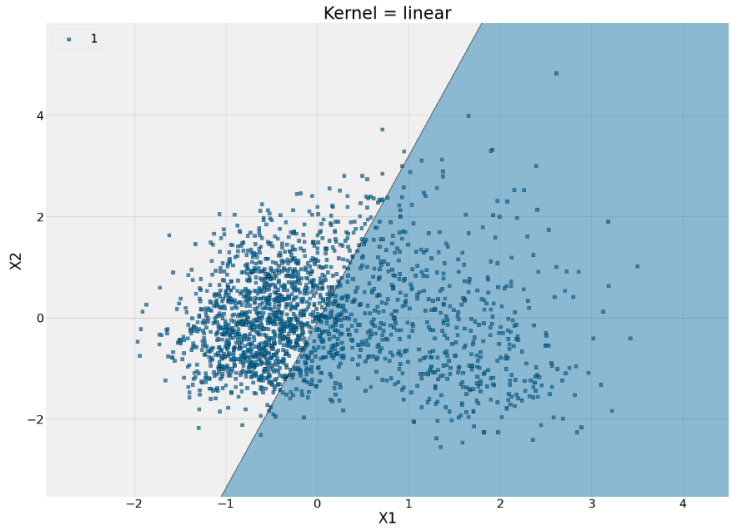


Figure 4.5: Decision boundary for One-Class SVM with kernel="linear".

- **"poly":** OC-SVM generates new features by applying the polynomial combination of all the existing features and as a result it generates a non-linear decision boundary using those polynomial features. Figure 4.6 shows the decision boundary obtained for a polynomial kernel.

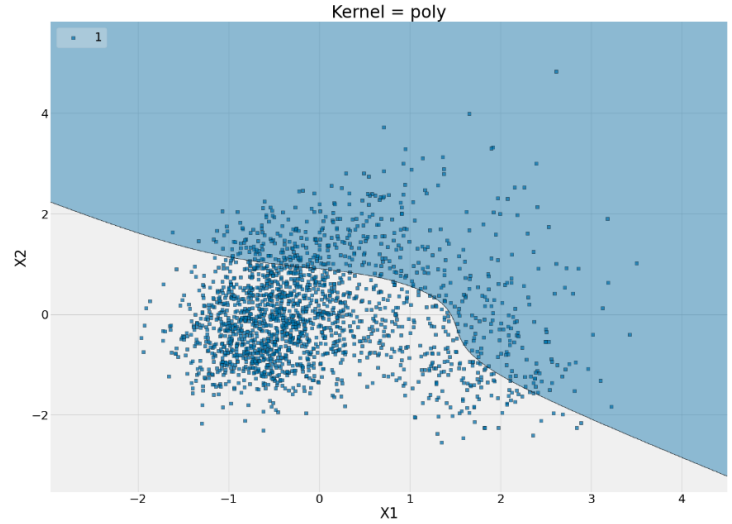


Figure 4.6: Decision boundary for One-Class SVM with kernel="poly".

- **"rbf":** OC-SVM uses gaussian curves around the data points, and sums these so that the decision boundary can be defined by a type of topology condition such as curves. The figure 4.7 shows the decision boundary obtained for a radial basis function kernel.

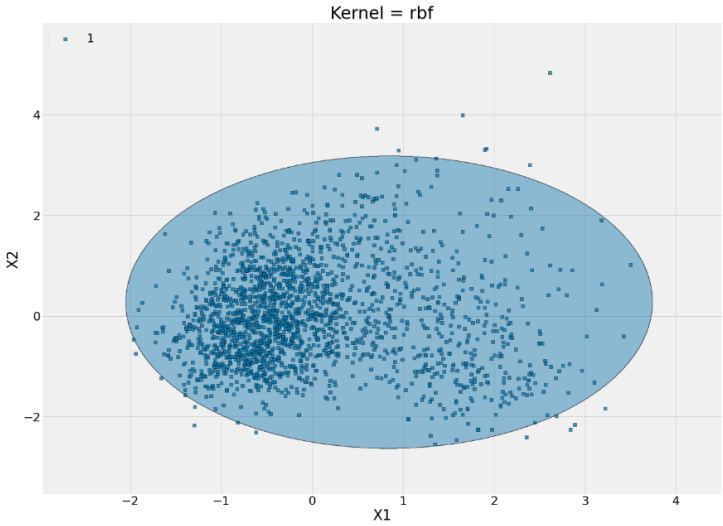


Figure 4.7: Decision boundary for One-Class SVM with kernel="rbf".

Because the decision boundary generated for OC-SVM with the rbf kernel includes almost all of the data points in the training dataset, it is the ideal kernel to utilize.

Hyperparameter Tuning

Tuning the hyperparameters consists in finding the best set of parameters for our model.

The hyperparameters for One-Class SVM are:

- **Nu:** means the proportion of outliers we expect in our data. Many unsupervised ML algorithms require us to know (or hint at) the number of outliers or class members we expect.
- **Gamma:** is a parameter of the rbf kernel. It controls the influence of individual training samples this effects the "smoothness" of the model. A low value improves the "smoothness" and "generalisability" of the model, while a high value reduces it but makes the model "tighter-fitted" to the training data.

As an alternative of hyperparameter tuning, we use Grid Search for predefined values of Nu and Gamma. Grid Search involves on performing multiple training runs with different parameter combinations and pick the set that returns the best metrical results. Table 4.2 shows the F1-score for validation dataset for different Nu and Gamma combinations.

Nu	0.0001	0.001	0.005	0.01	0.05	...	0.01	0.05	0.1	0.5	1
Gamma	0.0001	0.0001	0.0001	0.0001	0.0001	...	1	1	1	1	1
F1-score	0.94	0.94	0.94	0.94	0.66	...	0.03	0.03	0.03	0.03	0.03

Table 4.2: F1-score for validation dataset for different Nu and Gamma combinations

Figure 4.8 shows a 3D plot where x-axis is Nu, y-axis is Gamma and z-axis is F1-score for validation dataset.

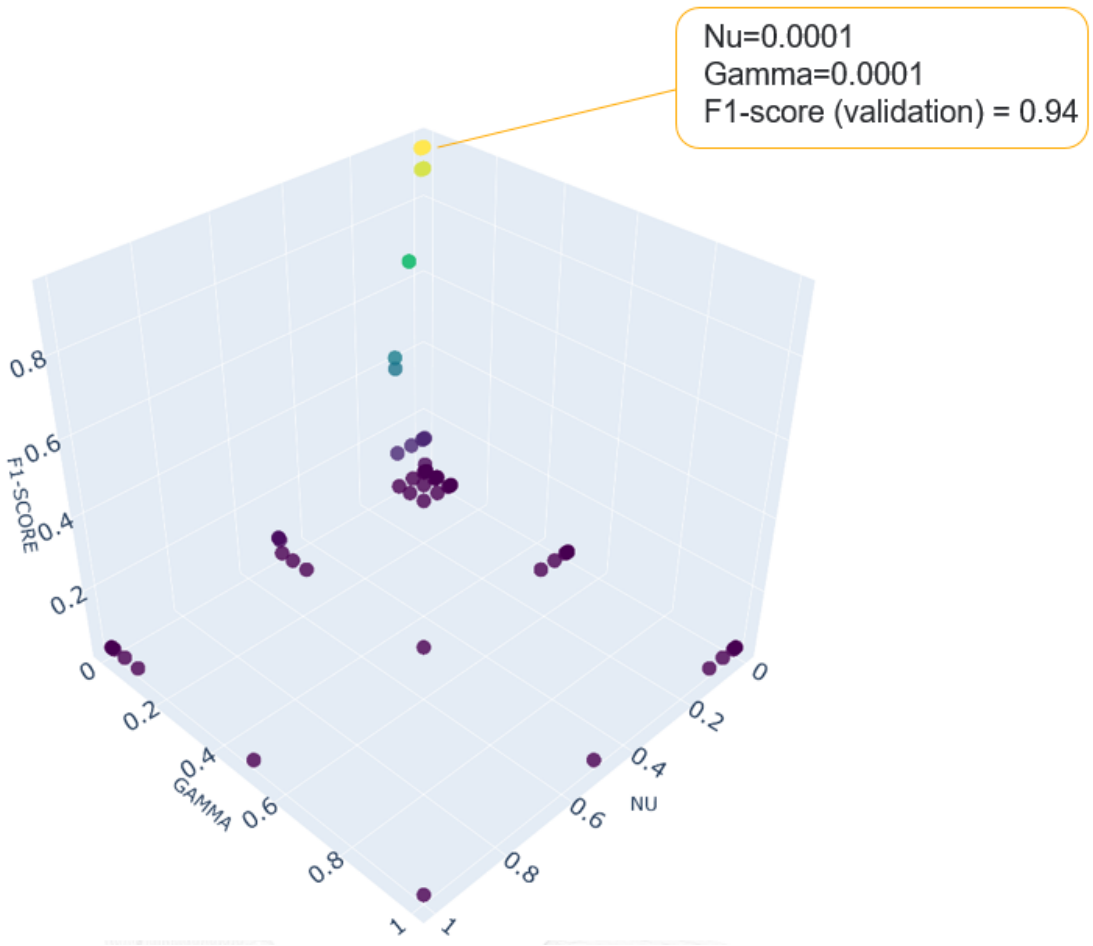


Figure 4.8: Variation of validation's F1-score for different nu and gamma combinations.

Since for $Gamma = Nu = 0.0001$ we obtain the greatest value for validation's F1-score, we choose then those parameters to train the One-Class SVM model.

4.3.2 Results

Running the experiment using the discussed settings gives the following confusion Matrix for test dataset:

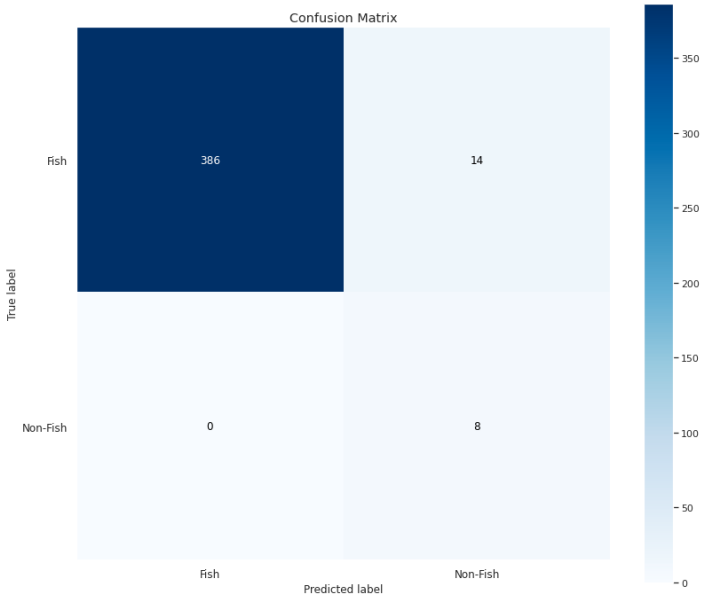


Figure 4.9: Confusion matrix for test dataset.

The confusion matrix indicates that the model did excellent job in predicting non-fish images and predict them all as outliers. However, it fails to predict 14 out of 400 fish images and classifies them as outliers (Non-fish images).

4.4 Fork Length prediction

4.4.1 Image Processing Approach

This approach is tested on 50 samples. Each image is resized to $(1024, 720, 3)$ before applying the whole process of the approach.

For the 50 samples, the Mean Absolute Percentage Error (MAPE) is equal to 8%. The figure 4.13 shows a 2D scatter plot where x-axis refers to actual fork length and y-axis refers to the predicted fork length for the 50 samples.

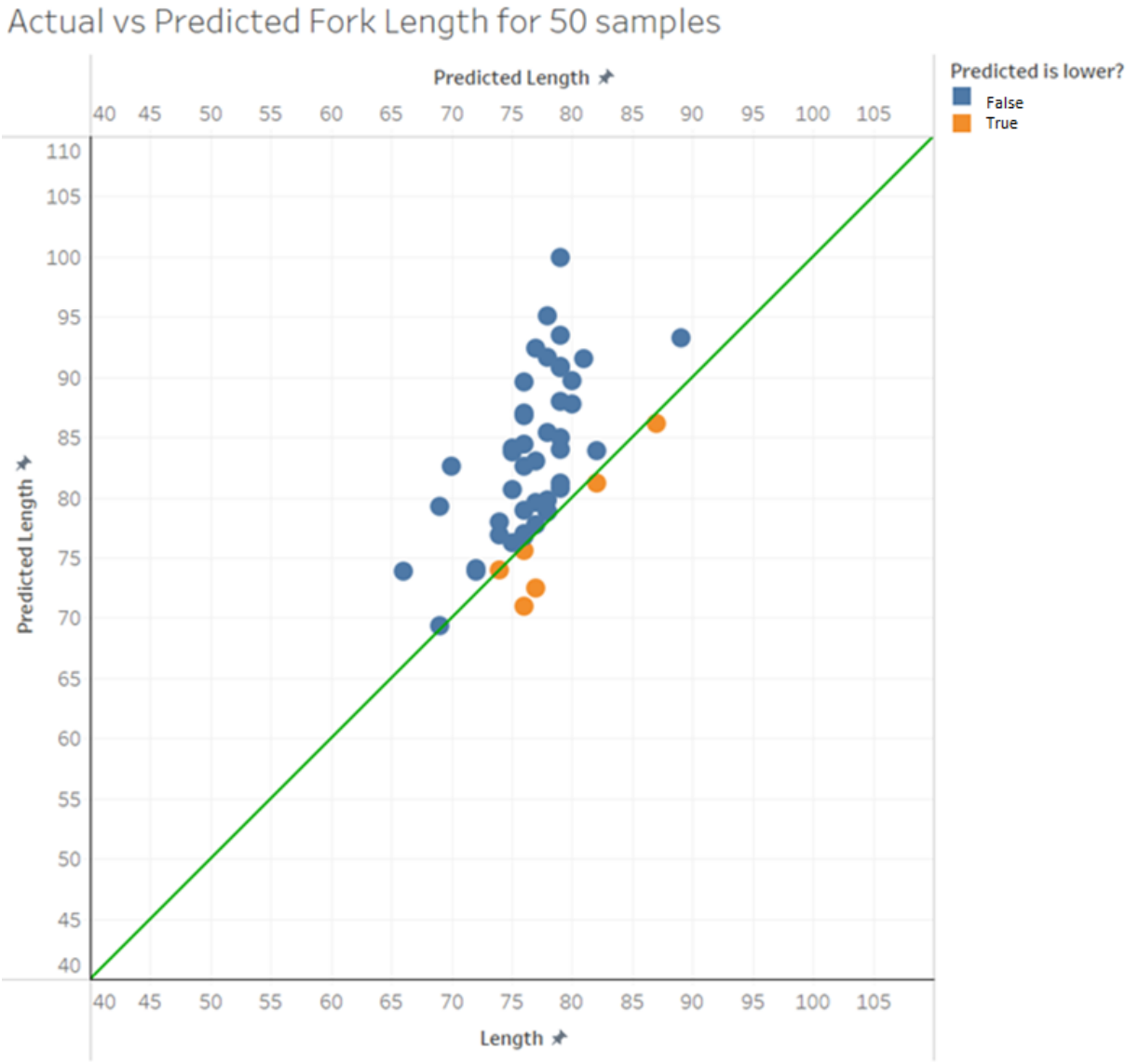


Figure 4.10: Actual vs Predicted Fork Length with Image Processing Approach for 50 samples.

This approach frequently predicts a fork length that is longer than the real one. This is due to the width of the predicted box which represents the fish's total length and is clearly greater than the fork length.

Figure 4.11 shows a visual representation of predicted and actual fork length in a fish image example.

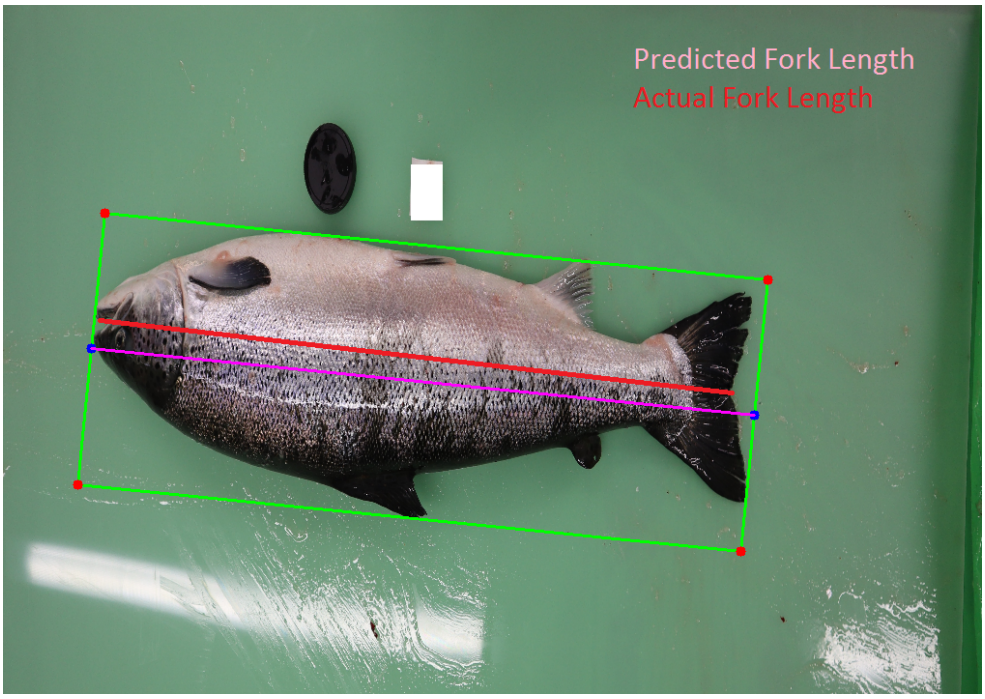


Figure 4.11: A visual representation of predicted and actual fork length in a fish image example.

Some images, as illustrated in figure 4.12, do not respond well to this approach. In fact, undesired objects such as the fisherman's hand or a piece of the fish are sometimes detected separately.

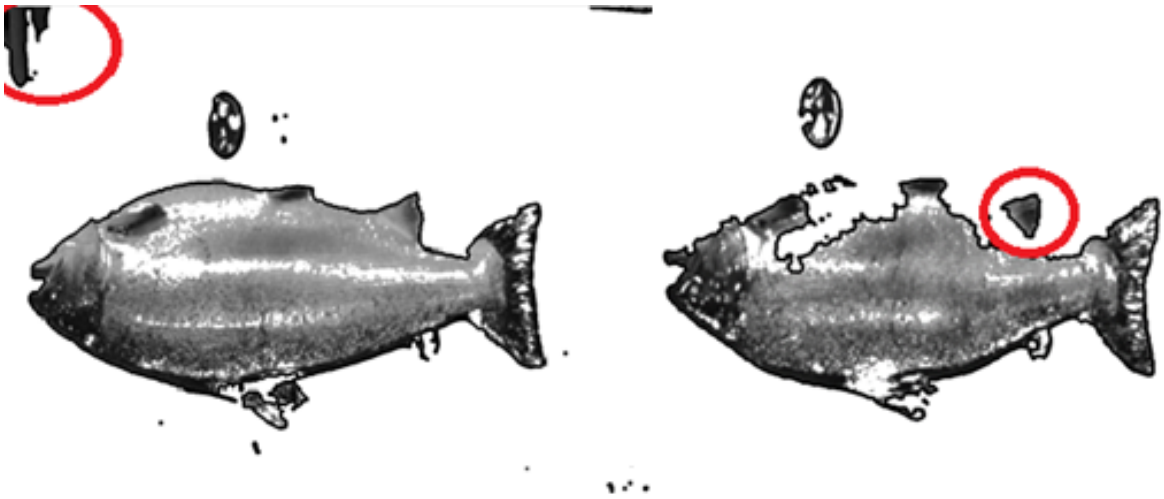


Figure 4.12: Unwanted objects detected in images.

4.4.2 Machine Learning Approach

Semantic Segmentation Settings and Results

At a first moment, we generate masks to train the U-Net model. However, creating masks is a time consuming task. That is why we create masks only

for 400 images.

Before running experiments, we resize those images to $(320, 480, 3)$ and we scale them using standardizing. We run three experiments and we use dice coefficient score to evaluate on both training and validation images. Table 4.3 details the specification and results of each experiment.

	Total images	**Splits**	**Epochs**	**Dice Score (Train - Valid)**
First Experiment	200	70% - 30%	10	0.9901 - 0.9898
Second Experiment	300	70% - 30%	10	0.9902 - 0.9901
Third Experiment	400	70% - 30%	10	0.9907 - 0.9824

Table 4.3: Experiment's specifications and results.

For the training dataset, the third experiment produces the best Dice Coefficient score. The gap between Dice Coefficient score for train and Dice Score Coefficient for validation, on the other hand, is the most significant. So it is possible that this is due to overfitting.

When comparing the Dice Coefficient scores for the train and validation experiments, we may conclude that the second experiment yielded best results. We select the trained U-Net model on 210 (total images including validation 300 images) images and examine the training and validation loss curves as shown in figure 4.13.

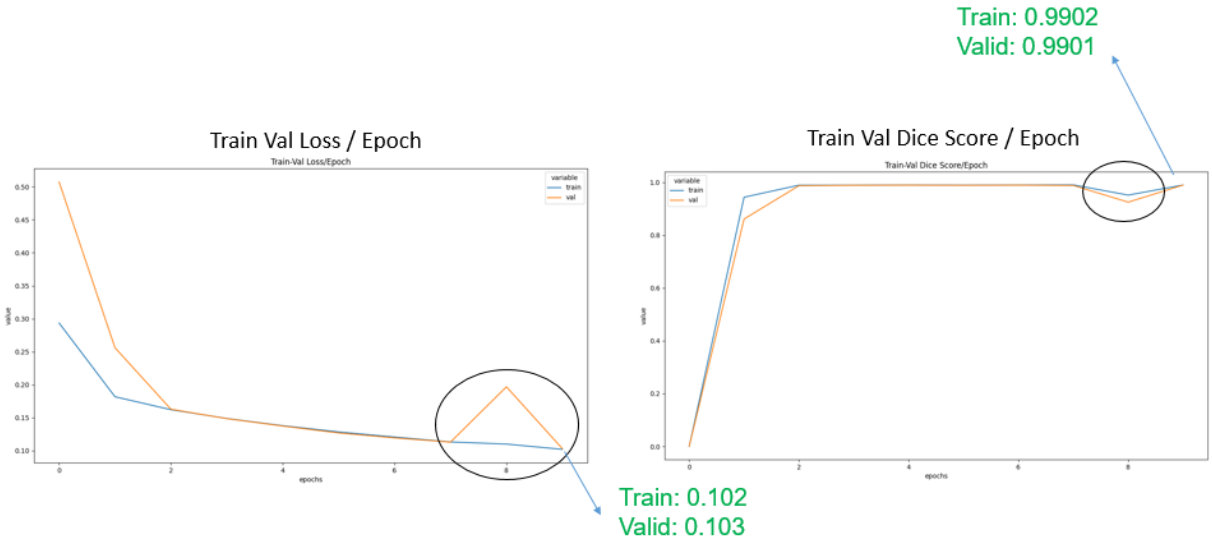


Figure 4.13: Training and validation loss curves

The loss for training and validation decreases during the first seven epochs, indicating that the model is learning. However, in the seventh epoch, the validation loss begins to rise, reaching 0.19, before decreasing again. This may

confirm the model's overfitting.

We examine various masks that have been predicted. The 210 photos used to train the model are made up of 207 photos with a fish looking to the right. Figure 4.14 shows how well the model predicts them. Furthermore, the training images are composed of 3 images of a fish looking to the left. As shown in figure 4.15, the model fails to provide effective mask predictions for those images.

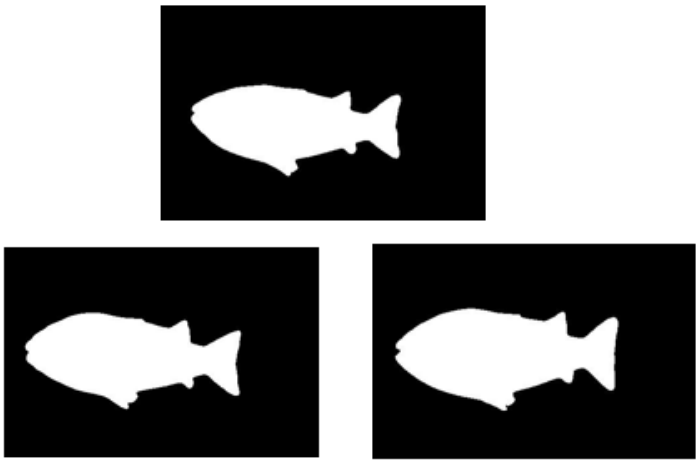


Figure 4.14: Examples of predicted masks for fishes looking to the left

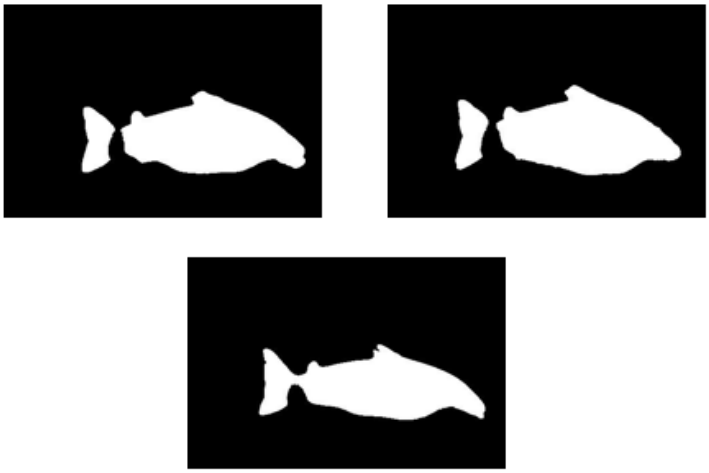


Figure 4.15: Examples of predicted masks for fishes looking to the right

We apply an image data augmentation technique to correct those predictions. In fact, we flip the training images horizontally so that the ratios for images with fish looking to the right and images with fish looking to the left are the same. We retrain the model, and we obtain a Dice Coefficient score equal to 0.9903 for train dataset and 0.9894 for validation dataset. In addi-

tion, the model now provides good predictions for both variations of images, as seen in figure 4.16.

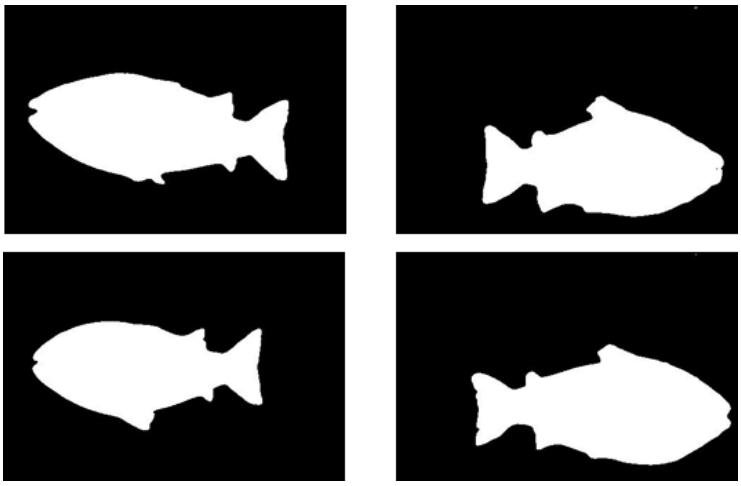


Figure 4.16: Examples of predicted masks

Regression Settings and Results

In this part, we use the predicted masks from the U-Net model refined using GNU Image Manipulation Program tool to create a segmented image.

We normalize those masks by dividing each pixel by 255. Then, as shown in figure 4.17, we simply multiply the mask and the corresponding original image pixel by pixel to obtain segmented image.



Figure 4.17: Example of a segmented image

As a result, we obtain 2608 segmented images, split 80% for training and 20% for testing. We use 25% as a validation dataset from the training images.

We resize all segmented images to $(227, 227, 3)$ and scale them using standardizing then we feed the training images to a pretrained **ResNet18** architecture.

We use the following settings to train the model:

- **Batch size:** 16.
- **Number of epochs:** 20.
- **Optimizer** is Adam.
- **Learning rate:** $5e-4$.

The MAPE value of the model is equal to 5.56% for train and 5.42% for validation. Figures 4.18 and 4.19 show a scatter plot of actual against predicted fork length for the train and test datasets, respectively.

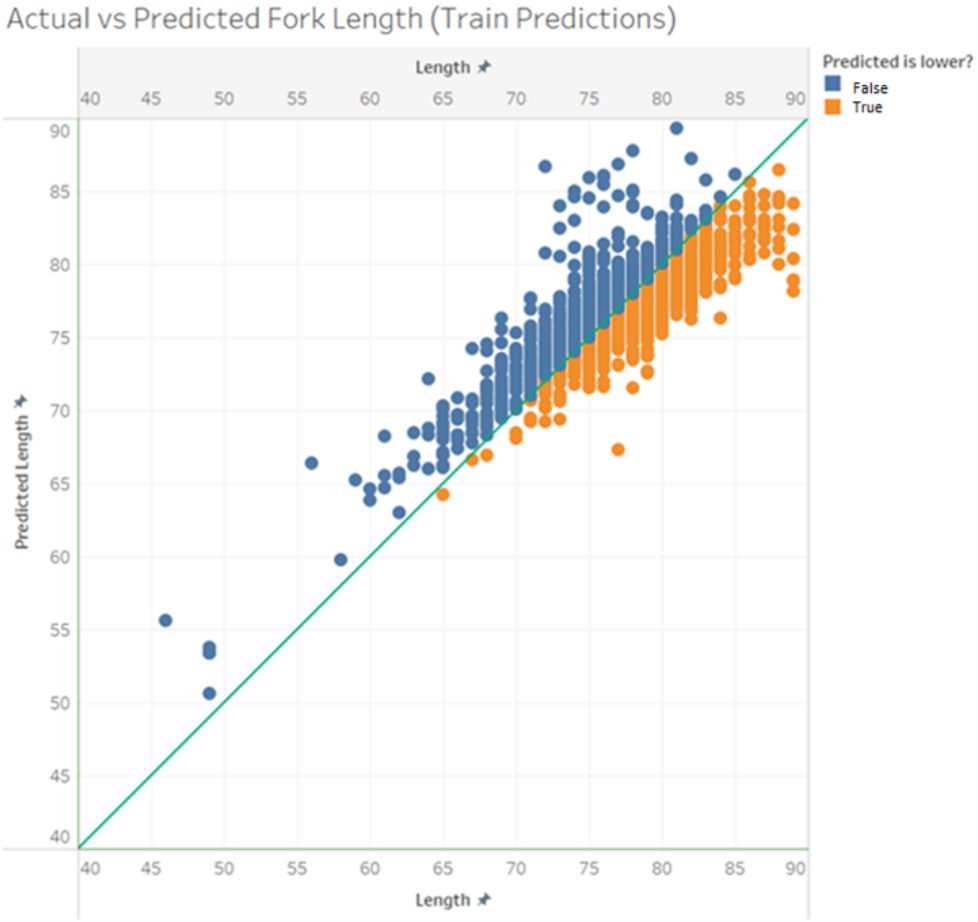


Figure 4.18: Actual vs Predicted fork length for train dataset.

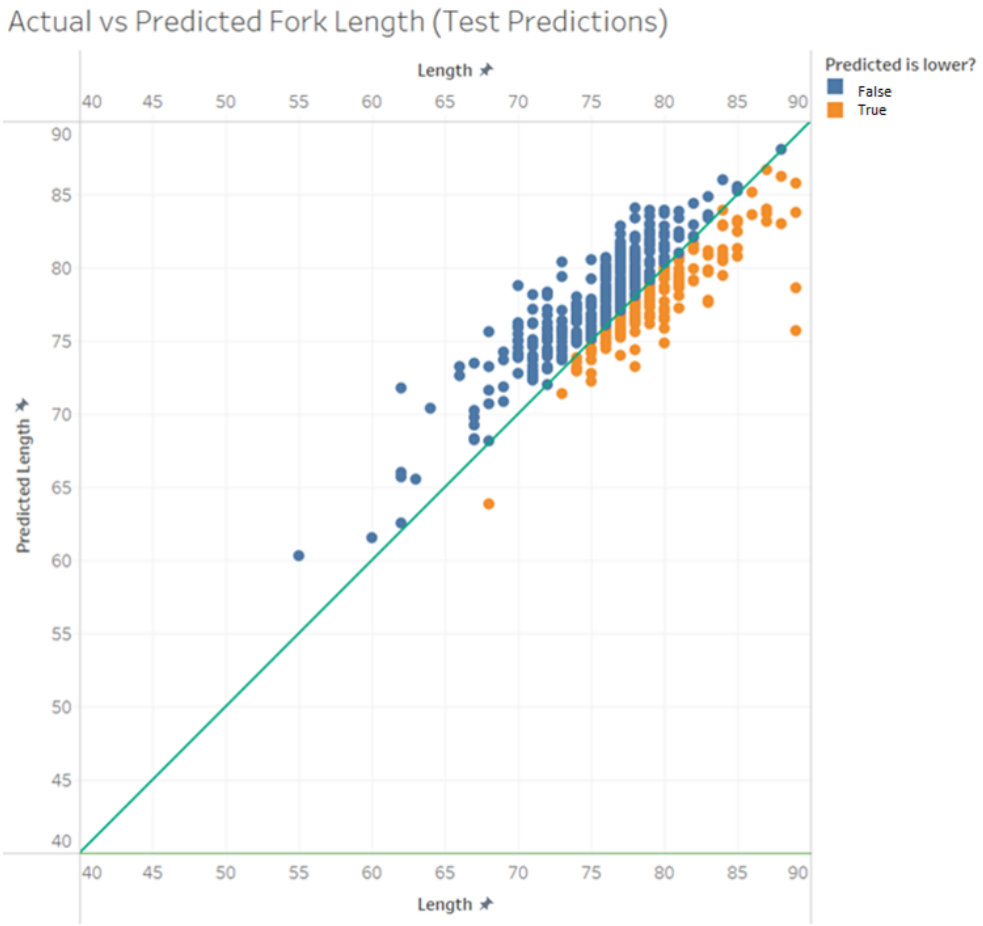


Figure 4.19: Actual vs Predicted fork length for test dataset.

The predicted values are very close to the real ones. Figures 4.18 and 4.19 show that as the actual fork length increases, the model is more likely to predict a value lower than the actual value.

4.4.3 Final Evaluation

We analyze the predictions for the two approaches on 50 samples to compare them. Figure 4.20 shows the Actual versus Predicted Fork Length for the 50 images. The blue dots that represent the predicted Fork Length of the Image Processing technique are frequently found above the red dots that represent the predicted Fork Length of the Machine Learning approach. Furthermore, the red dots are closer to the red line that corresponds to $x = y$ than the blue dots.

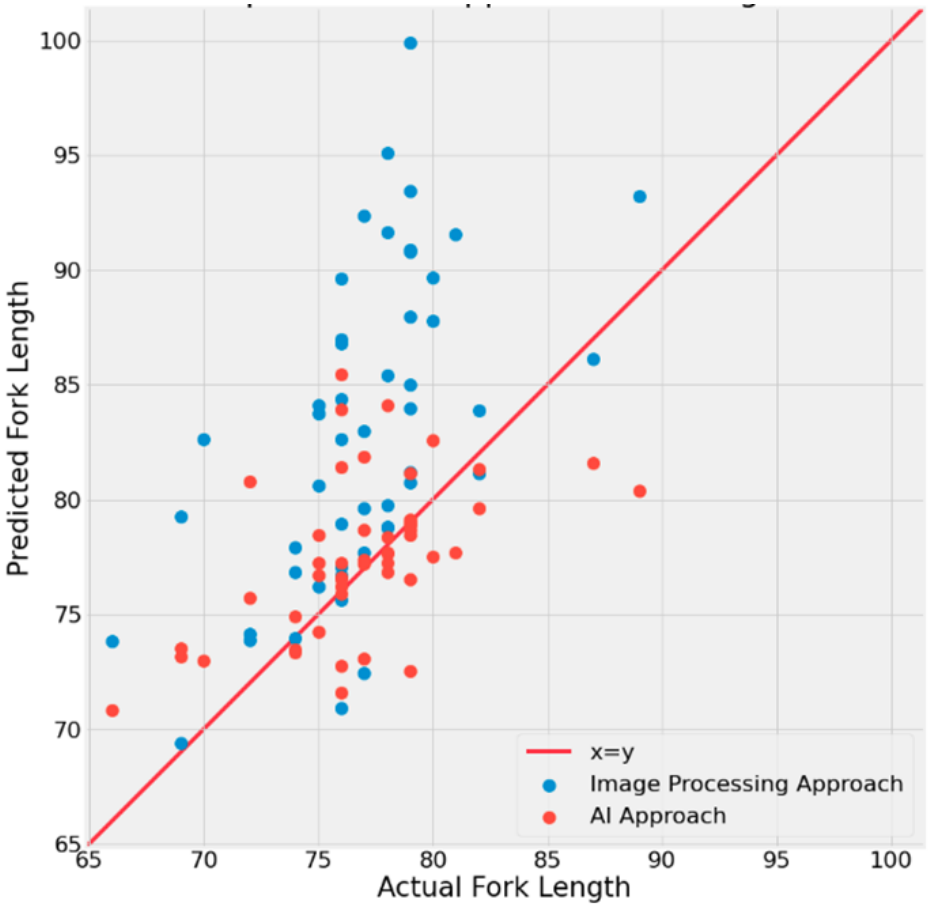


Figure 4.20: Predictions for 50 samples with Machine Learning and Image Processing Approach

For those 50 images, the MAPE value for Image Processing Approach is equal to 8%. For the Machine Learning approach, however, it is equal to 3%.

As a conclusion, we can confirm that Machine Learning outperforms the Image Processing approach.

4.5 Sex Prediction

Settings

We use 2524 segmented images, we resize them to $(227, 227, 3)$ and we scaled them using standardizing. Since the dataset is imbalanced, we use random oversampling to balance the data. We get then 4360 as total images where 2180 refers to female fishes and 2180 to male fishes. We split the dataset into 70% for training, 15% for validation and 15% for testing. To choose the best pretrained CNN architecture, we need to do various experiments.

We use the following settings for all the experiments:

- **Batch size:** 8.
- **Number of epochs:** 5.
- **Optimizer** is Adam.
- **Learning rate:** $1e-4$.
- **Unfreeze** last 5 **layers** and retrain them.

Results

We fit the dataset into three pretrained architectures: **ResNet18**, **ResNet50** and **ResNet152**.

The table 4.4 details the results of each experiment.

	Train Accuracy	**Validation Accuracy**
ResNet18	0.9993	0.9832
ResNet50	0.9640	0.8777
ResNet152	0.9617	0.9495

Table 4.4: Experiment's specifications and results.

Figures 4.21 and 4.22 show the variation of train and validation accuracy respectively for each epoch for ResNet18, ResNet50 and ResNet152.

Other architectures are outperformed by ResNet18. This model is then used to make predictions for test images. The confusion matrix for the test dataset is shown in figure 4.23.

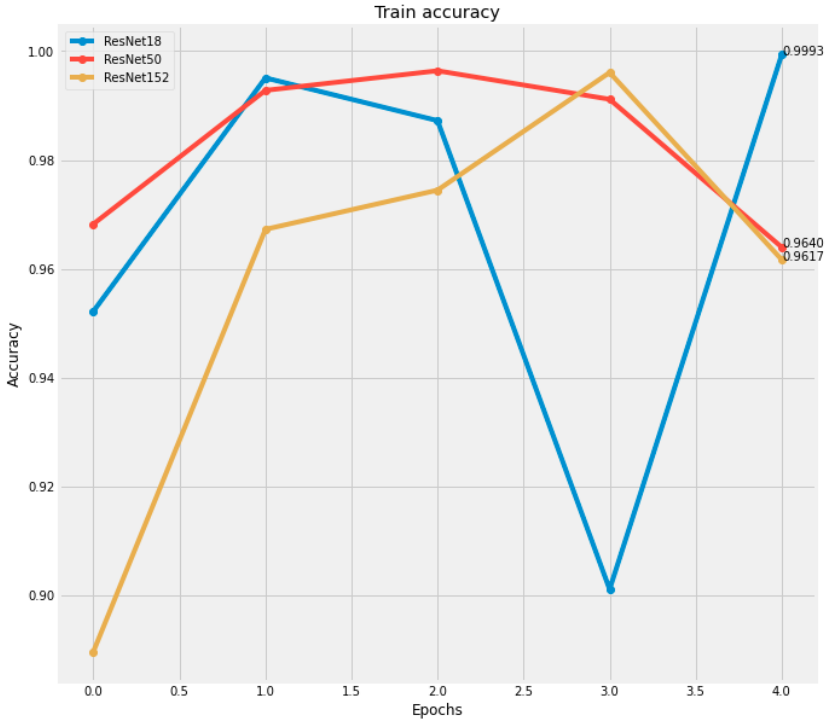


Figure 4.21: Train accuracy.

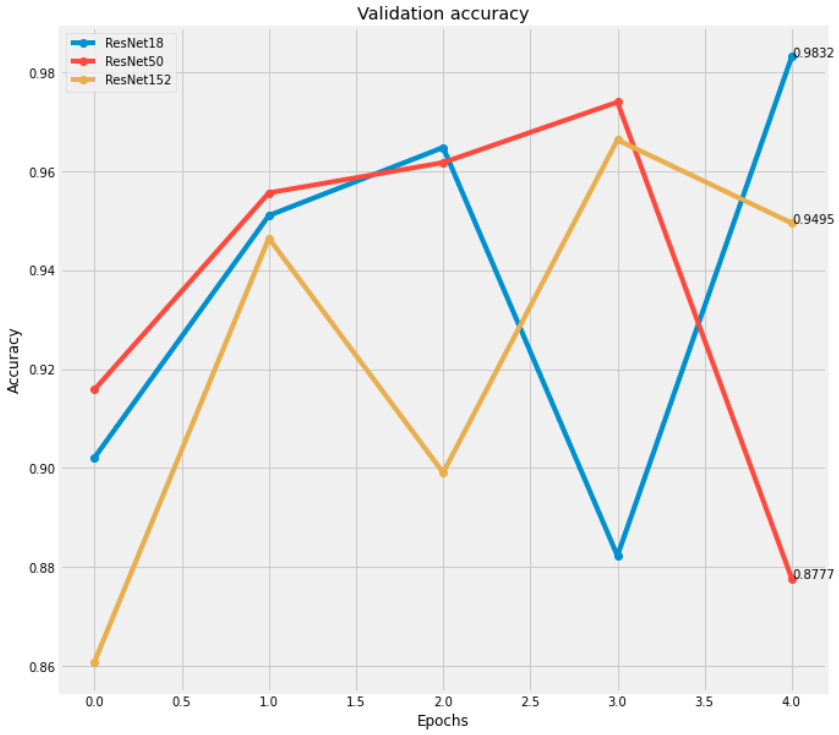


Figure 4.22: Validation accuracy.

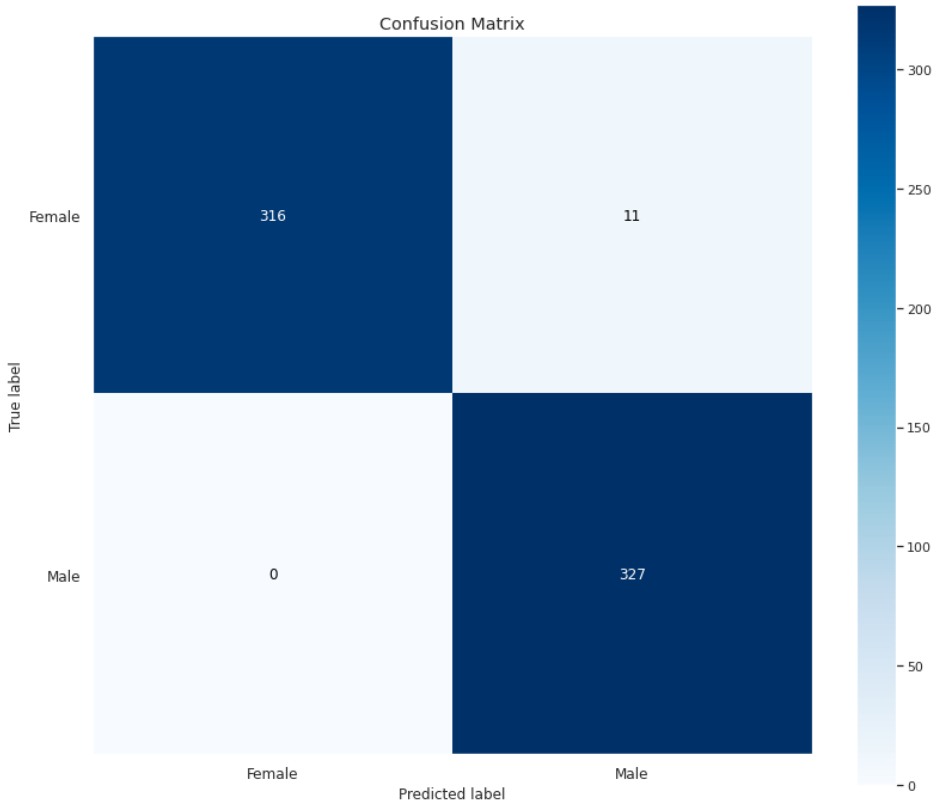


Figure 4.23: Confusion Matrix for test dataset.

The model gives us an $accuracy = 0.9831$ for test dataset. All male fishes are accurately predicted by the model. It does not, however, predict 11 female fishes. For the test dataset, we get $accuracy = 0.9831$.

4.6 Weight prediction

4.6.1 Settings

The dataset is a csv file containing 2492 rows and 3 columns. The columns are:

- **Image Name** which refers to the name of the image.
- **Fork Length** associated fork length.
- **Sex** associated gender.
- **Weight** associated weight.

We split the dataset into 80% for training and 20% for testing. We try different boosting algorithms such as **Gradient Boosting Regressor**, **XGBoost Regressor** and **LightGBM Regressor** to predict the weight of the fish. The settings for Gradient Boosting Regressor are:

- **n_estimators:** 3000
- **learning_rate:** 0.05
- **min_samples_leaf:** 15

The settings used to train LightGBM Regressor are:

- **n_estimators:** 6000
- **learning_rate:** 0.01
- **max_bin:** 55
- **bagging_fraction:** 0.8

The settings used to train XGBoost Regressor are:

- **n_estimators:** 6000
- **learning_rate:** 0.01
- **max_depth:** 4
- **gamma:** 0.6
- **subsample:** 0.7

4.6.2 Results

The table 4.5 details the results of each experiment.

	Train MAPE	**Test MAPE**
Gradient Boosting Regressor	0.064%	0.082%
LightGBM Regressor	0.065%	0.083%
XGBoost Regressor	0.063%	0.079%

Table 4.5: Results of experiments.

The figure 4.24 shows a barplot which illustrates a comparison between Gradient Boosting, LightGBM and XGBoost models.

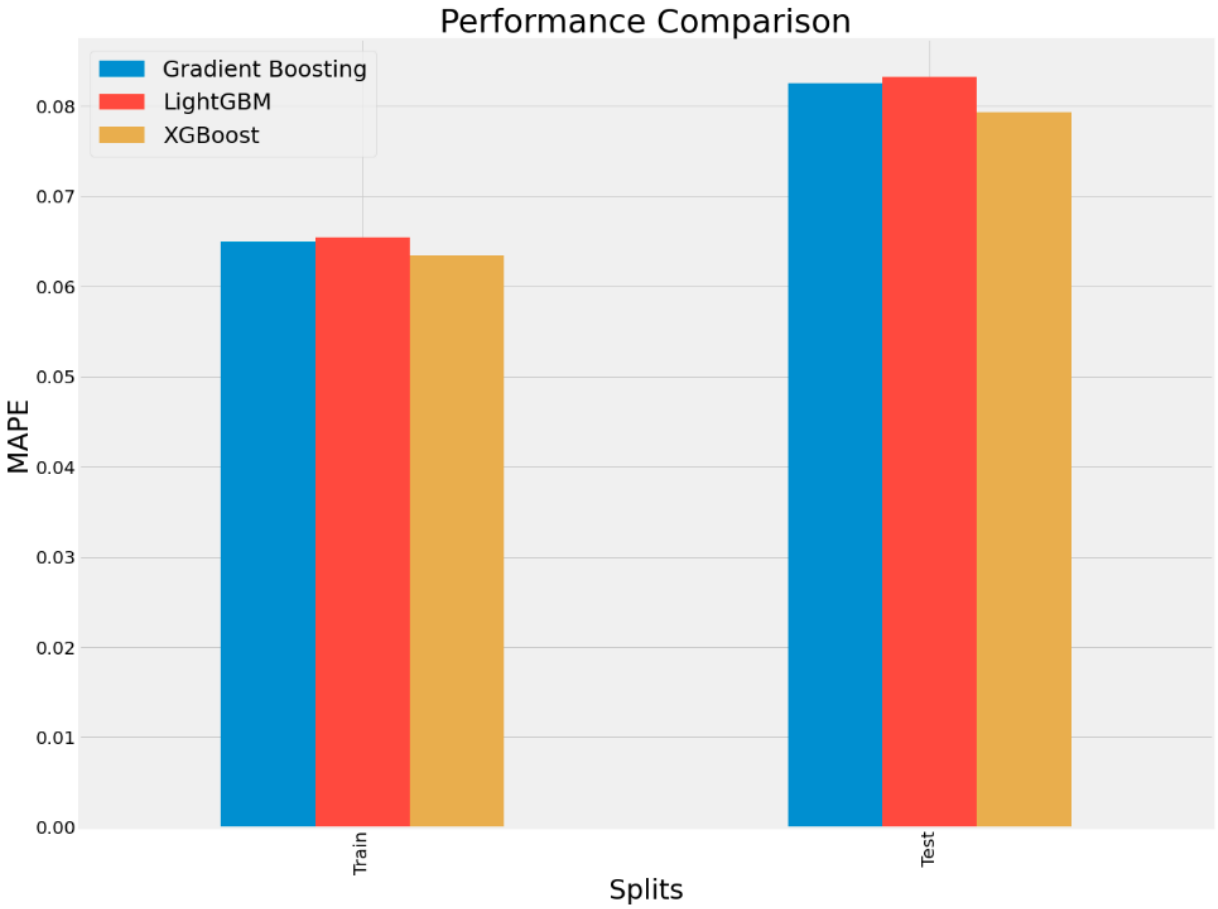


Figure 4.24: Comparison between Gradient Boosting, LightGBM and XGBoost models.

Other models are outperformed by XGBoost. The predicted weights for the test dataset are then visualized using this model as shown in the figures 4.25 and 4.26.

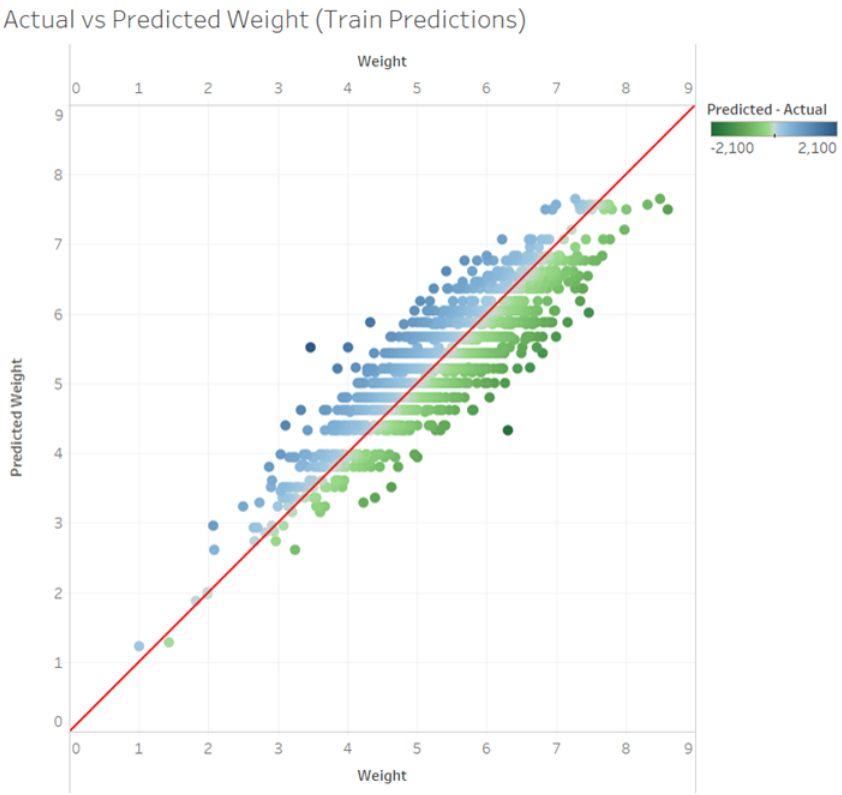


Figure 4.25: Actual vs Predicted weight for train dataset.

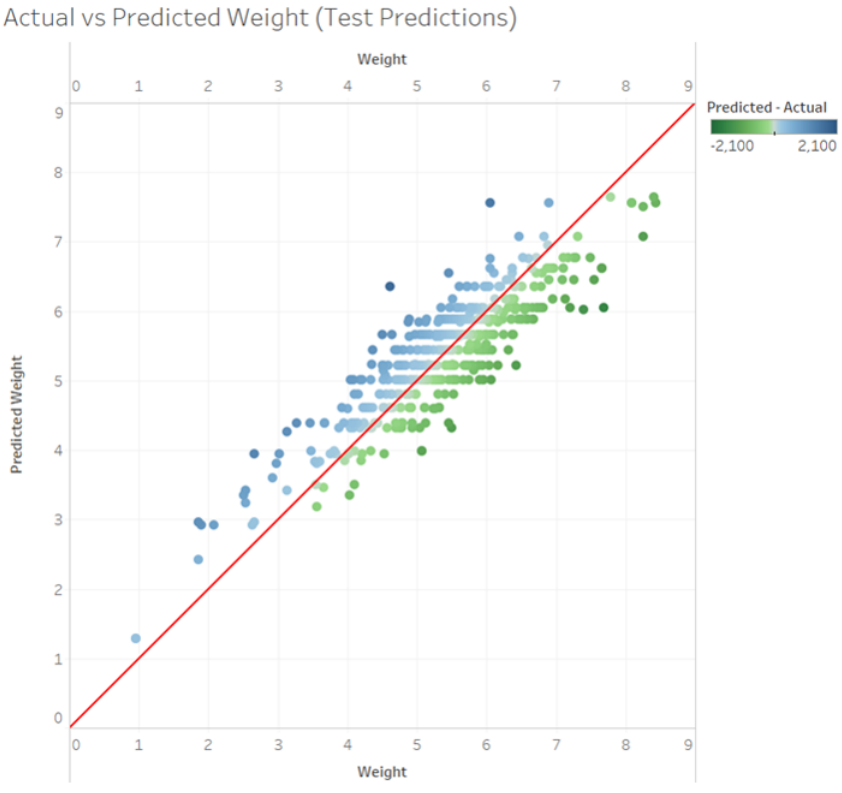


Figure 4.26: Actual vs Predicted weight for test dataset.

The predicted values are quite similar to the actual values. In fact, the gap between predicted and actual values for the train dataset might be lower than 2 and lower than 1.7 for the test dataset.

4.7 Conclusion

During this chapter, we have successfully implemented a full pipeline that takes an input image and predicts the weight of the fish (if the image contains a fish) by predicting its fork length and its gender. For predicting the fork length, we have used two approaches: one based on machine learning techniques and the other on traditional computer vision techniques. We have concluded that the Machine Learning approach outperforms the Image Processing approach after comparing their predictions on 50 samples. For predicting the gender, we have tested multiple CNN architectures and we have found that ResNet18 outperformed other models. Finally, we have performed weight prediction experiments using several tree-based models and XGBoost outperformed all other models that have been tested.

General Conclusion

In this project, an automated measurement method for the fork length and the weight in images was developed. This method can replace manual measurements using a tape measure and a weight scale and thus speeding up the process of tracking the fish's growth.

This work has achieved the goals it has set, that is build a full pipeline for automated weight prediction of the fish from its image by predicting the fork length and the gender.

During this project, We have built an outlier detection model to skip non-fish images from the rest of the process. Then, we have built two approaches to perform fork length prediction. The Machine Learning approach which is based on image segmentation and CNN regressor outperformed the Image Processing approach which is based on traditional computer vision techniques such as object detection using Canny Edge Detection algorithm. Next, we have built an architecture to predict the gender of the fish. Finally, using the predicted fork length and predicted sex, we have used a tree based model to predict the weight.

We achieve very good scores for:

- Fork length prediction: $MAPE = 5\%$.
- Sex prediction: $Accuracy = 0.98$.
- Weight prediction: $MAPE = 0.06\%$.

However, it is still open for improvements. For example, we can generalize the work for fish under the water. To do so, we will need to gather images for fish beneath water in different positions to add variety to the dataset and help the model generalize better. Furthermore, we can expand this work to build a real time prediction system.

Bibliography

[1] Cs231n convolutional neural networks for visual recognition. https://cs231n.github.io/neural-networks-1/#nn, [Accessed: March 15, 2021].

[2] Loss functions. https://ml-cheatsheet.readthedocs.io/en/latest/loss_functions.html, [Accessed: March 15, 2021].

[3] Ching-Lu Hsieh Chi-Hsuan Tseng and Yan-Fu Kuo. Automatic measurement of the body length of harvested fish using convolutional neural networks. 189:36–47, 2020.

[4] Nina Danielsen. Simple image classification with resnet-50. https://shorturl.at/bhGTW, [Accessed: March 24, 2021].

[5] Jocelyn D'Souza. A quick guide to boosting in ml. https://medium.com/greyatom/a-quick-guide-to-boosting-in-ml-acf7c1585cb5, [Accessed: April 6, 2021].

[6] Garima. One class svm(oc-svm). https://medium.com/@mail.garima7/one-class-svm-oc-svm-9ade87da6b10, [Accessed: March 3, 2021].

[7] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. Deep learning. https://www.deeplearningbook.org/, [Accessed: March 15, 2021].

[8] Prince Grover. Gradient boosting from scratch. https://blog.mlreview.com/gradient-boosting-from-scratch-1e317ae4587d, [Accessed: April 6, 2021].

[9] Zakaria Jaadi. A step-by-step explanation of principal component analysis (pca). https://shorturl.at/lFH06, [Accessed: March 30, 2021].

[10] Cory Maklin. Gradient boosting decision tree algorithm explained. https://shorturl.at/dCIY2, [Accessed: April 6, 2021].

[11] Diksha Kurchaniya Mohd. Aquib Ansari and Manish Dixit. A comprehensive analysis of image edge detection techniques. *International Journal of Multimedia and Ubiquitous Engineering*, 12(11):1–12, 2017.

[12] Rosely Kumoi Norhaida Abdullah Mohd Rahim Mohd Shafry, Amjad Rehman and Tanzila Saba. Filedi framework for measuring fish length from digital images. 7:607—618, 2012.

[13] Mohammed Zeeshan Mulla. Cost, activation, loss function|| neural network|| deep learning. what are these? https://shorturl.at/lINR2, [Accessed: March 15, 2021].

[14] Philipp Fischer Olaf Ronneberger and Thomas Brox. U-net: Convolutional networks for biomedical image. 2015.

[15] Prabhu. Understanding of convolutional neural network (cnn) — deep learning. https://shorturl.at/zEHO2, [Accessed: March 15, 2021].

[16] Richard Kinh Gian Do Rikiya Yamashita, Mizuho Nishio and Kaori Togashi. Convolutional neural networks: an overview and application in radiology. 9:611—629, 2018.

[17] Mingxing Tan and Quoc V. Le. Efficientnet: Rethinking model scaling for convolutional neural networks. 2020.

[18] David M.J. Tax and Robert P.W. Duin. Uniform object generation for optimizing one-class classifiers. *Journal of Machine Learning Research 2 (2001)*, 2:155–173, 2002.

[19] Avinash Sharma V. Understanding activation functions in neural networks. shorturl.at/cxDR2, [Accessed: March 15, 2021].

[20] Roemer Vlasveld. Introduction to one-class support vector machines. https://cutt.ly/wcP8HQi, [Accessed: March 3, 2021].